

Practical 1

Project: Movie Collection Manager

Objective: Work with arrays and objects to manage a collection of movies dynamically.

Tasks:

- Create an array of movie objects, where each object has fields like title, genre, rating, and release year.
- Write functions to:
 - Add a new movie to the collection.
 - List all movies in a specific genre.
 - Find the highest-rated movie in the collection.
 - Use `map()` to create a list of all movie titles.
 - Use `filter()` to find movies released after a specific year.
 - Log the results to the console using template literals.

Screenshots of Code:

```

JS indexjs X
practical_1 > JS indexjs > ...
1  let movieCollection = [
2    {
3      title: "The House Next Door",
4      genre: "Horror",
5      rating: 8.8,
6      releaseYear: 2010
7    },
8    {
9      title: "The Dark Knight",
10     genre: "Action",
11     rating: 9.0,
12     releaseYear: 2008
13    },
14    {
15     title: "The Exorcist",
16     genre: "Sci-Fi",
17     rating: 8.7,
18     releaseYear: 2000
19    },
20    {
21     title: "The Godfather",
22     genre: "Crime",
23     rating: 9.2,
24     releaseYear: 1972
25    },
26    {
27     title: "Interstellar",
28     genre: "Sci-Fi",
29     rating: 8.6,
30     releaseYear: 2014
31    }
32  ];
33  function addMovie(title, genre, rating, releaseYear) {
34    const newMovie = {

```

```

JS indexjs X
practical_1 > JS indexjs > ...
33  function addMovie(title, genre, rating, releaseYear) {
34    console.log(`Movie ${title} added to the collection.`);
35  }
36  function listMoviesByGenre(genre) {
37    const moviesInGenre = movieCollection.filter(movie => movie.genre === genre);
38    console.log(`Movies in the "${genre}" genre:`);
39    moviesInGenre.forEach(movie => {
40      console.log(`${movie.title} (${movie.releaseYear}) - Rating: ${movie.rating}`);
41    });
42  }
43  function findHighestRatedMovie() {
44    const highestRatedMovie = movieCollection.reduce((highest, current) => {
45      return (current.rating > highest.rating) ? current : highest;
46    });
47    console.log(`The highest-rated movie is "${highestRatedMovie.title}" with a rating of ${highestRatedMovie.rating}.`);
48  }
49  function getMovieTitles() {
50    const movieTitles = movieCollection.map(movie => movie.title);
51    console.log("All movie titles in the collection:");
52    movieTitles.forEach(title => console.log(title));
53  }
54  function getMoviesReleasedAfter(year) {
55    const recentMovies = movieCollection.filter(movie => movie.releaseYear > year);
56    console.log(`Movies released after ${year}:`);
57    recentMovies.forEach(movie => {
58      console.log(`${movie.title} (${movie.releaseYear})`);
59    });
60  }
61  addMovie("Criminal Justice", "Drama", 8.9, 2019);
62  listMoviesByGenre("Sci-Fi");
63  findHighestRatedMovie();
64  getMovieTitles();
65  getMoviesReleasedAfter(2000);

```

Screenshots of Output:

```

C:\Program Files\nodejs\node.exe .\movie-collection.js
IN Bollywood Movie Collection Manager 🎬
📁 Added "3 Idiots" to the collection!
📁 Romance Movies:
- Dilwale Dulhania Le Jayenge (1995)

🏆 Highest Rated Movie:

🌟 Movie Details 🌟
Title: Sholay
Genre: Action
Rating: 9.2/10
Release Year: 1975
Director: Ramesh Sippy
Lead Actors: Amitabh Bachchan, Dharmendra

📋 All Movie Titles:
> (4) ['Sholay', 'Dilwale Dulhania Le Jayenge', 'Lagaan', '3 Idiots']

🕒 Movies after 2000:

🌟 Movie Details 🌟
Title: Lagaan
Genre: Historical Drama
Rating: 8.5/10
Release Year: 2001
Director: Ashutosh Gowariker
Lead Actors: Aamir Khan

🌟 Movie Details 🌟
Title: 3 Idiots
Genre: Comedy-Drama
Rating: 8.4/10
Release Year: 2009
Director: Rajkumar Hirani
Lead Actors: Aamir Khan, R. Madhavan, Sharman Joshi

Collection: Desi Cinema Treasures
Total Movies: 4

```

Conclusion: This practical taught us how to organize movie information using arrays and objects in JavaScript. We learned to add movies, search through them by genre, find the highest-rated movies, and create lists of movie titles. This helps understand how to organize and search through collections of information.

Practical 2

Project: Task Reminder System

Objective: Build a task reminder system that schedules reminders and handles asynchronous delays.

Tasks

- Create an array of task objects, where each task has fields like title, dueTime (in minutes from now), and priority.
- Write functions to:
 - Add a new task to the list.
 - Sort tasks by their priority.
 - Display tasks due within a certain timeframe (e.g., next 10 minutes).
 - Simulate sending reminders using setTimeout() based on the task's dueTime.
- Use try...catch to handle errors, such as invalid task data or missing fields.
- Export and import the task management functions using JavaScript modules.

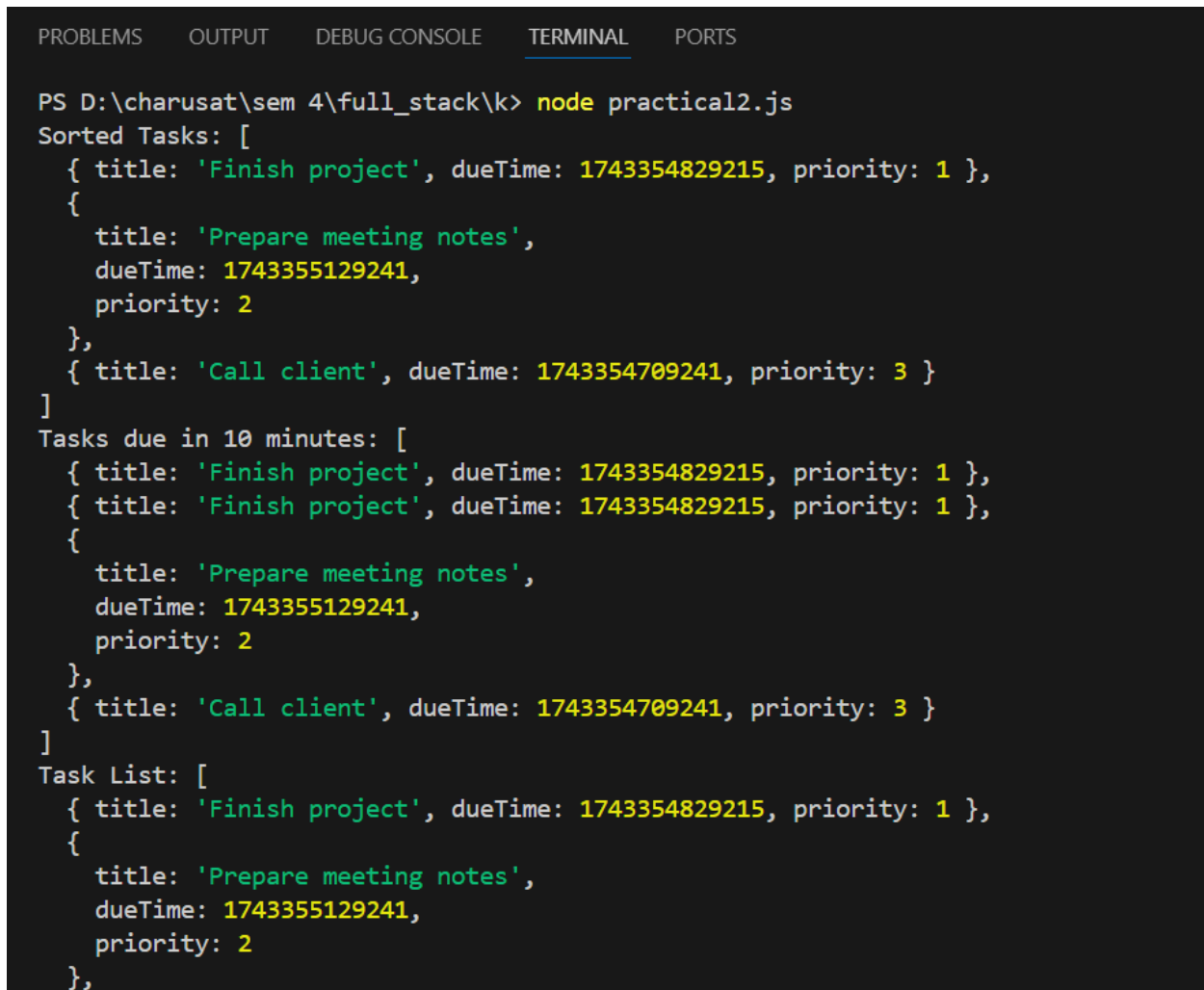
Screenshots of Code:

```

1  class TaskManager {
2      constructor() {
3          this.tasks = [];
4      }
5
6      addTask(title, dueTime, priority) {
7          try {
8              if (!title || typeof dueTime !== 'number' || !priority) {
9                  throw new Error("Invalid task data. Title, dueTime, and priority are required.");
10             }
11             const task = {
12                 title,
13                 dueTime: Date.now() + dueTime * 60000,
14                 priority
15             };
16             this.tasks.push(task);
17             this.scheduleReminder(task);
18         } catch (error) {
19             console.error("Error adding task:", error.message);
20         }
21     }
22
23     sortTasksByPriority() {
24         this.tasks.sort((a, b) => a.priority - b.priority);
25     }
26
27     getTasksDueWithin(minutes) {
28         const now = Date.now();
29         return this.tasks.filter(task => (task.dueTime - now) <= minutes * 60000);
30     }
31
32     scheduleReminder(task) {
33         const delay = task.dueTime - Date.now();
34         if (delay > 0) {
35             setTimeout(() => {
36                 console.log(`Reminder: Task '${task.title}' is due now!`);
37             }, delay);
38         }
39     }
40
41     displayTasks() {
42         console.log("Task List:", this.tasks);
43     }
44 }
45 const taskManager = new TaskManager();
46 taskManager.addTask("Finish project", 5, 1);
47 taskManager.addTask("Prepare meeting notes", 10, 2);
48 taskManager.addTask("Call client", 3, 3);
49 taskManager.sortTasksByPriority();
50 console.log("Sorted Tasks:", taskManager.tasks);
51 console.log("Tasks due in 10 minutes:", taskManager.getTasksDueWithin(10));
52 taskManager.displayTasks();

```

Screenshots of Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\charusat\sem 4\full_stack\k> node practical2.js
Sorted Tasks: [
  { title: 'Finish project', dueTime: 1743354829215, priority: 1 },
  {
    title: 'Prepare meeting notes',
    dueTime: 1743355129241,
    priority: 2
  },
  { title: 'Call client', dueTime: 1743354709241, priority: 3 }
]
Tasks due in 10 minutes: [
  { title: 'Finish project', dueTime: 1743354829215, priority: 1 },
  { title: 'Finish project', dueTime: 1743354829215, priority: 1 },
  {
    title: 'Prepare meeting notes',
    dueTime: 1743355129241,
    priority: 2
  },
  { title: 'Call client', dueTime: 1743354709241, priority: 3 }
]
Task List: [
  { title: 'Finish project', dueTime: 1743354829215, priority: 1 },
  {
    title: 'Prepare meeting notes',
    dueTime: 1743355129241,
    priority: 2
  },
]
```

Conclusion: In this practical, we built a system that reminds users about tasks they need to complete. We learned how to schedule tasks, organize them by priority, and show which tasks are due soon. This teaches us how computers can help us remember important things at the right time.

Practical 3

Project: System Utility Dashboard

- **Objective:** Create a command-line tool that provides system information.

- **Tasks:**

1. Use the os module to display the OS type, total memory, free memory, and CPU details.
2. Use the fs module to save the system information to a log file.
3. Use the path module to ensure the file is saved in a standardized format (logs/system-info.txt).

Screenshots of Code:

```

practical_3 > <> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>System Utility Dashboard</title>
7  <style>
8  body {
9    font-family: Arial, sans-serif;
10   margin: 0;
11   padding: 0;
12   background: #f4f4f4;
13   color: #333;
14 }
15 .container {
16   max-width: 800px;
17   margin: 50px auto;
18   padding: 20px;
19   background: #fff;
20   border-radius: 8px;
21   box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
22 }
23 h1 {
24   text-align: center;
25   color: #444;
26 }
27 #system-info {
28   margin-top: 20px;
29 }
30 .system-item {
31   margin: 10px 0;
32   padding: 10px;
33   background: #f9f9f9;
34   border: 1px solid #ddd;

```

```

<> index.html X
practical_3 > <> index.html > html > head > style > .system-item h3
2  <html lang="en">
3  <head>
7  <style>
30 .system-item {
32   padding: 10px;
33   background: #f9f9f9;
34   border: 1px solid #ddd;
35   border-radius: 5px;
36 }
37 .system-item h3 {
38   margin: 0;
39   font-size: 1.1em;
40   color: #555;
41 }
42 .system-item p {
43   margin: 5px 0 0;
44   color: #666;
45 }
46 </style>
47 </head>
48 <body>
49 <div class="container">
50 <h1>System Utility Dashboard</h1>
51 <div id="system-info">
52 <!-- Dynamic system information will appear here -->
53 </div>
54 </div>
55 <script>
56 // JSON data inside JavaScript
57 const systemInfo = {
58   "Operating System": "Windows 10",
59   "OS Version": "10.0.19041",
60   "Architecture": "64-bit",
61   "CPU": "Intel Core i7-10750H",

```

```
index.html X
practical_3 > index.html > html > head > style > .system-item h3
2   <html lang="en">
48  <body>
55  <script>
57  const systemInfo = {
59    OS version : "10.0.19041",
60    "Architecture": "64-bit",
61    "CPU": "Intel Core i7-10750H",
62    "Total Memory": "16384 MB",
63    "Free Memory": "4096 MB",
64    "Home Directory": "C:\\Users\\JohnDoe",
65    "Uptime": "3 days 5 hours 20 minutes"
66  };
67  // Function to display system information
68  function displaySystemInfo(data) {
69    const systemInfoDiv = document.getElementById('system-info');
70    systemInfoDiv.innerHTML = ''; // Clear existing content
71    Object.entries(data).forEach(([key, value]) => {
72      const item = document.createElement('div');
73      item.className = 'system-item';
74      item.innerHTML = `
75      <h3>${key}</h3>
76      <p>${value}</p>
77      `;
78      systemInfoDiv.appendChild(item);
79    });
80  }
81  // Call the function to display data
82  displaySystemInfo(systemInfo);
83  </script>
84  </body>
85  </html>
86
```

Screenshots of Output:

```
System Information:
-----
osType: Windows_NT
osPlatform: win32
osArchitecture: x64
osRelease: 10.0.19044
totalMemory: 16.00 GB
freeMemory: 8.45 GB
cpuModel: Intel(R) Core(TM) i7-10700K CPU @ 3.80GHz
cpuCores: 8
uptime: 10.25 hours
```

Conclusion: This practical showed us how to create a tool that gives information about a computer's system, like its memory and processor. We also learned how to save this information to a file for later. This helps understand how to gather and store system information.

Practical 4

Project: Basic Static File Server

Objective: Build an HTTP server to serve static files like HTML, CSS, and images.

Tasks:

- Create a simple HTML page with a welcome message and a few images.
- Use the http and fs modules to serve these files when the browser requests them.
- Add logic to handle 404 errors when a file is not found.

Screenshots of Code:

```
Prc3_1.html X 404_page.html # style.css JS server.js
practical_4 > Prc3_1.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Static File Server</title>
7 <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10 <h1>Welcome to the Static File Server</h1>
11 <div class="images">
12 
13 
14 </div>
15 </body>
16 </html>

practical_4 > 404_page.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>404 - File Not Found</title>
7 <style>
8 body {
9 font-family: Arial, sans-serif;
10 text-align: center;
11 background-color: #f8d7da;
12 color: #721c24;
13 }
14 h1 {
15 font-size: 2.5rem;
16 }
17 p {
18 font-size: 1.2rem;
19 }
20 </style>
21 </head>
22 <body>
23 <h1>404 - File Not Found</h1>
24 <p>The requested file could not be found on this server.</p>
25 </body>
26 </html>
```

```

Prc3_1.html 404_page.html # style.css JS server.js
practical_4 > # style.css > .images img
1  body {
2    font-family: Arial, sans-serif;
3    text-align: center;
4    background-color: #f4f4f9;
5    color: #333;
6    margin: 0;
7    padding: 0;
8  }
9  h1 {
10   margin-top: 20px;
11   font-size: 2.5rem;
12   color: #2c3e50;
13 }
14 p {
15   font-size: 1.2rem;
16   margin: 10px 0 30px;
17 }
18 .images {
19   display: flex;
20   justify-content: center;
21   gap: 20px;
22   margin-top: 20px;
23 }
24 .images img {
25   width: 200px;
26   height: auto;
27   border: 2px solid #ccc;
28   border-radius: 8px;
29   box-shadow: 2px 2px 8px rgba(0, 0, 0, 0.2);
30 }

practical_4 > JS server.js > ...
1  const http = require('http');
2  const fs = require('fs');
3  const path = require('path');
4  const PORT = 3000;
5  // MIME types for common file extensions
6  const mimeTypes = {
7    '.html': 'text/html',
8    '.css': 'text/css',
9    '.js': 'application/javascript',
10   '.jpg': 'image/jpeg',
11   '.png': 'image/png',
12   '.ico': 'image/x-icon',
13 };
14 // Create the HTTP server
15 const server = http.createServer((req, res) => {
16   let filePath = `${req.url}`;
17   if (filePath === '/') {
18     filePath = './index.html';
19   }
20   const extname = String(path.extname(filePath)).toLowerCase();
21   const contentType = mimeTypes[extname] || 'application/octet-stream';
22   fs.readFile(filePath, (err, content) => {
23     if (err) {
24       if (err.code === 'ENOENT') {
25         // File not found
26         fs.readFile('./404_page.html', (error, page404) => {
27           res.writeHead(404, { 'Content-Type': 'text/html' });
28           res.end(page404 || '404: File Not Found', 'utf-8');
29         });
30       } else {
31         // Server error
32         res.writeHead(500);
33         res.end(`500: Internal Server Error\n${err.code}`, 'utf-8');
34       }
35     }
36   });
37 });

```

```
21 const contentType = mimeTypes[extname] || 'application/octet-stream';
22 fs.readFile(filePath, (err, content) => {
23   if (err) {
24     if (err.code === 'ENOENT') {
25       // File not found
26       fs.readFile('./404_page.html', (error, page404) => {
27         res.writeHead(404, { 'Content-Type': 'text/html' });
28         res.end(page404 || '404: File Not Found', 'utf-8');
29       });
30     } else {
31       // Server error
32       res.writeHead(500);
33       res.end(`500: Internal Server Error\n${err.code}`, 'utf-8');
34     }
35   } else {
36     // Serve the file
37     res.writeHead(200, { 'Content-Type': contentType });
38     res.end(content, 'utf-8');
39   }
40 });
41 });
42 // Start the server
43 server.listen(PORT, () => {
44   console.log(`Server is running on http://localhost:${PORT}`);
45 });
```

Screenshots of Output:

Conclusion: We built a simple web server that can show webpages, images, and other files when someone visits our website. We also learned how to handle errors when a file can't be found. This helps understand how websites deliver content to visitors.

Practical 5

Project: File Organizer Tool

Objective: Build a CLI tool to organize files into folders based on their type (e.g., images, documents, videos).

Tasks:

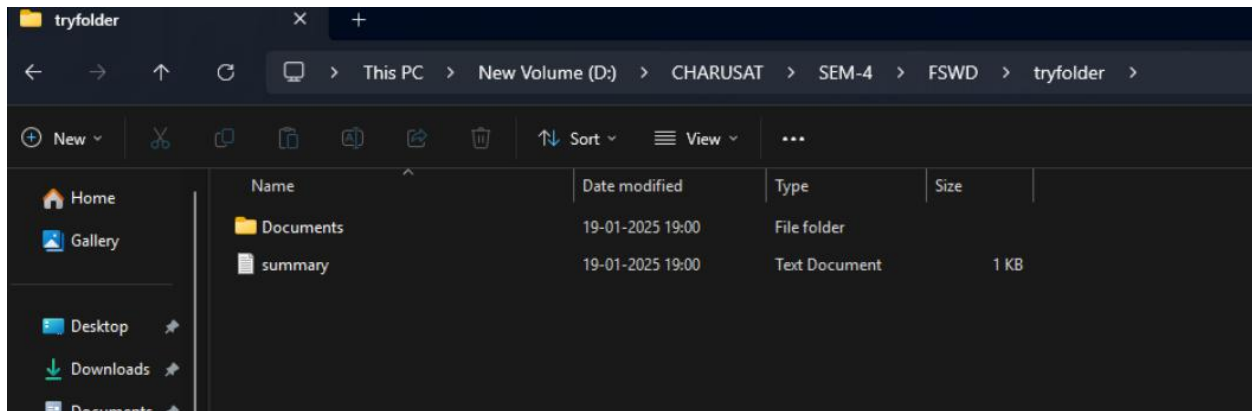
- Accept a directory path as an input from the user.
- Use the fs module to read all files in the directory.
- Move files into folders like Images, Documents, and Others based on their extensions.
- Log the operations performed into a summary.txt file.

Screenshots of Code:

```
1  import fs from 'fs';
2  import path from 'path';
3
4  const organizeFiles = (directoryPath) => {
5    if (!fs.existsSync(directoryPath)) {
6      console.log("Invalid directory path");
7      return;
8    }
9
10   const categories = {
11     Images: ['.jpg', '.jpeg', '.png', '.gif'],
12     Documents: ['.pdf', '.docx', '.txt', '.xlsx'],
13     Videos: ['.mp4', '.avi', '.mov'],
14     Others: []
15   };
16
17   const files = fs.readdirSync(directoryPath);
18   let logData = '';
19
20   files.forEach(file => {
21     const ext = path.extname(file).toLowerCase();
22     let folder = 'Others';
23
24     for (const category in categories) {
25       if (categories[category].includes(ext)) {
26         folder = category;
27         break;
28       }
29     }
30
31     const folderPath = path.join(directoryPath, folder);
32     if (!fs.existsSync(folderPath)) {
33       fs.mkdirSync(folderPath);
34     }
35   });
36 }
```

```
23
24     for (const category in categories) {
25         if (categories[category].includes(ext)) {
26             folder = category;
27             break;
28         }
29     }
30
31     const folderPath = path.join(directoryPath, folder);
32     if (!fs.existsSync(folderPath)) {
33         fs.mkdirSync(folderPath);
34     }
35
36     const oldPath = path.join(directoryPath, file);
37     const newPath = path.join(folderPath, file);
38     fs.renameSync(oldPath, newPath);
39     logData += `Moved ${file} to ${folder}\n`;
40 });
41
42 fs.writeFileSync(path.join(directoryPath, 'summary.txt'), logData);
43 console.log("Files organized successfully!");
44 };
45
46 const directoryPath = process.argv[2];
47 if (directoryPath) {
48     organizeFiles(directoryPath);
49 } else {
50     console.log("Please provide a directory path");
51 }
```

Screenshots of Output:



Conclusion: This practical taught us how to create a tool that automatically sorts files into folders based on what type they are (like images or documents). This helps understand how to organize files on a computer automatically.

Practical 6

Project: User Management System

Objective: Create a RESTful API to manage user data without using Express.js.

Tasks:

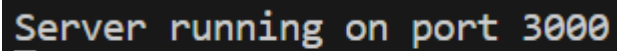
- Implement the following endpoints using the http module:
 - GET /users: Return a list of all users stored in a JSON file.
 - POST /users: Accept new user data in the request body and add it to the JSON file.
 - DELETE /users/:id: Remove a user by their ID from the JSON file.
 - Use the fs module to store and retrieve user data persistently.
 - Test the API using Postman or curl.

Screenshots of Code:

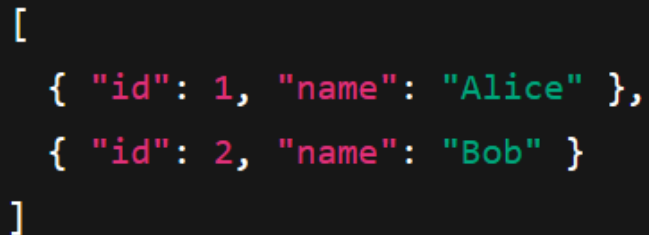
```

1  const http = require('http');
2  const fs = require('fs');
3  const path = require('path');
4  const usersFile = path.join(__dirname, 'users.json');
5  function getUsers() {
6      if (!fs.existsSync(usersFile)) return [];
7      return JSON.parse(fs.readFileSync(usersFile, 'utf8'));
8  }
9  function saveUsers(users) {
10     fs.writeFileSync(usersFile, JSON.stringify(users, null, 2));
11 }
12 const server = http.createServer((req, res) => {
13     if (req.method === 'GET' && req.url === '/users') {
14         const users = getUsers();
15         res.writeHead(200, { 'Content-Type': 'application/json' });
16         res.end(JSON.stringify(users));
17     } else if (req.method === 'POST' && req.url === '/users') {
18         let body = '';
19         req.on('data', chunk => { body += chunk.toString(); });
20         req.on('end', () => {
21             try {
22                 const newUser = JSON.parse(body);
23                 const users = getUsers();
24                 newUser.id = users.length ? users[users.length - 1].id + 1 : 1;
25                 users.push(newUser);
26                 saveUsers(users);
27                 res.writeHead(201, { 'Content-Type': 'application/json' });
28                 saveUsers(users);
29                 res.writeHead(201, { 'Content-Type': 'application/json' });
30                 res.end(JSON.stringify(newUser));
31             } catch (err) {
32                 res.writeHead(400, { 'Content-Type': 'text/plain' });
33                 res.end('Invalid JSON');
34             }
35         });
36     } else if (req.method === 'DELETE' && req.url.startsWith('/users/')) {
37         const id = parseInt(req.url.split('/')[2]);
38         let users = getUsers();
39         const filteredUsers = users.filter(user => user.id !== id);
40         if (users.length === filteredUsers.length) {
41             res.writeHead(404, { 'Content-Type': 'text/plain' });
42             res.end('User not found');
43         } else {
44             saveUsers(filteredUsers);
45             res.writeHead(200, { 'Content-Type': 'text/plain' });
46             res.end('User deleted');
47         }
48     } else {
49         res.writeHead(404, { 'Content-Type': 'text/plain' });
50         res.end('Not Found');
51     }
52 });
53 const PORT = 3000;
54 server.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

Screenshots of Output:A terminal window with a black background and light green text. The text reads "Server running on port 3000". A white cursor is visible on the line below the text.

```
Server running on port 3000
```

A terminal window with a black background and light green text. The text displays a JSON array containing two objects: one for Alice (id: 1) and one for Bob (id: 2).

```
[  
  { "id": 1, "name": "Alice" },  
  { "id": 2, "name": "Bob" }  
]
```

Conclusion: We built a system that can store, show, add, and remove user information. This teaches us how websites manage user data and how to build simple APIs (ways for computers to talk to each other).

Practical 7

Project: Simple Content Delivery Server

Objective: Create a static file server with basic APIs for logging user visits.

Tasks:

- Serve static HTML, CSS, and JS files from a public directory.
- Log every user visit (IP, time) to a visits.log file using middleware.
- Provide an API endpoint (GET /logs) to retrieve the log data as JSON.

Screenshots of Code:

```
const express = require('express');
const fs = require('fs');
const path = require('path');
const app = express();
const port = 3000;
app.use((req, res, next) => {
  const logEntry = `${new Date().toISOString()} - IP: ${req.ip}\n`;
  fs.appendFile('visits.log', logEntry, (err) => {
    if (err) {
      console.error('Error writing to log file:', err);
    }
  });
  next();
});
app.use(express.static(path.join(__dirname, 'public')));
app.get('/logs', (req, res) => {
  fs.readFile('visits.log', 'utf8', (err, data) => {
    if (err) {
      return res.status(500).json({ error: 'Could not read log file' });
    }
    const logs = data.trim().split('\n').map(entry => {
      const [timestamp, ip] = entry.split(' - IP: ');
      return { timestamp, ip };
    });
    res.json(logs);
  });
});
app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

Screenshots of Output:

```
Pretty-print ☐  
[{"timestamp": "2025-03-31T07:22:32.151Z", "ip": "::1"}, {"timestamp": "2025-03-31T07:22:53.861Z", "ip": "::1"}]
```

Conclusion: This practical showed us how to create a server that delivers website content and keeps track of who visits. This helps understand how websites track visitors and serve content.

Practical 8

Project: E-Commerce Product Catalog API**Objective:** Build an API for managing an e-commerce product catalog.**Tasks:**

- GET /products: Return all products.
- GET /products/:id: Fetch a specific product by ID.
- GET /products?category=electronics: Filter products by category.
- Use route parameters and query strings effectively.

Screenshots of Code:

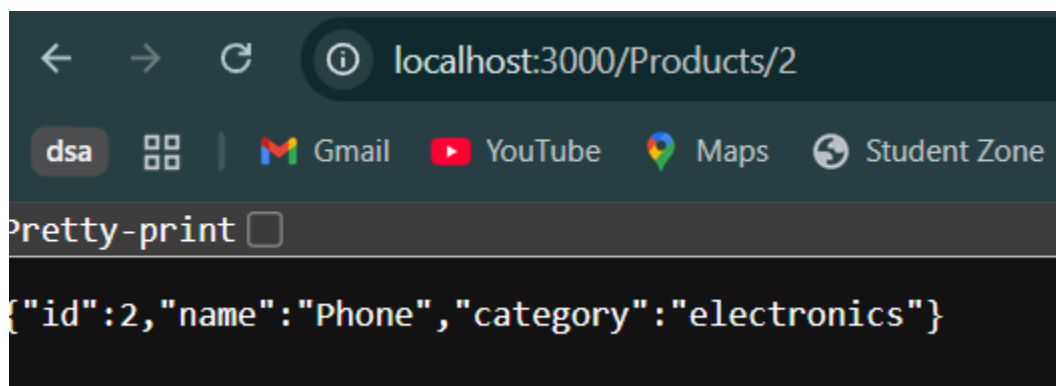
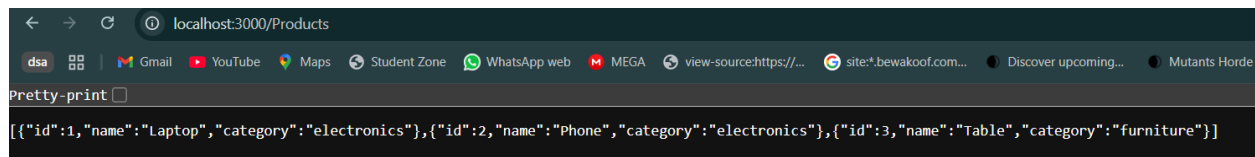
```
1  const express = require('express');
2  const fs = require('fs');
3  const path = require('path');
4  const app = express();
5  const port = 3000;
6  app.use(express.json());
7  app.use((req, res, next) => {
8    const logEntry = `${new Date().toISOString()} - IP: ${req.ip}\n`;
9    fs.appendFile('visits.log', logEntry, (err) => {
10      if (err) {
11        console.error('Error writing to log file:', err);
12      }
13    });
14    next();
15  });
16  app.use(express.static(path.join(__dirname, 'public')));
17  app.get('/logs', (req, res) => {
18    fs.readFile('visits.log', 'utf8', (err, data) => {
19      if (err) {
20        return res.status(500).json({ error: 'Could not read log file' });
21      }
22      const logs = data.trim().split('\n').map(entry => {
23        const [timestamp, ip] = entry.split(' - IP: ');
24        return { timestamp, ip };
25      });
26      res.json(logs);
27    });
28  });
29  const products = [
30    { id: 1, name: 'Laptop', category: 'electronics' },
31    { id: 2, name: 'Phone', category: 'electronics' }
```

```

25     });
26     res.json(logs);
27   });
28 });
29 const products = [
30   { id: 1, name: 'Laptop', category: 'electronics' },
31   { id: 2, name: 'Phone', category: 'electronics' },
32   { id: 3, name: 'Table', category: 'furniture' }
33 ];
34 const productRouter = express.Router();
35 productRouter.get('/', (req, res) => {
36   const { category } = req.query;
37   if (category) {
38     return res.json(products.filter(product => product.category === category));
39   }
40   res.json(products);
41 });
42 productRouter.get('/:id', (req, res) => {
43   const product = products.find(p => p.id === parseInt(req.params.id));
44   if (!product) {
45     return res.status(404).json({ error: 'Product not found' });
46   }
47   res.json(product);
48 });
49 app.use('/products', productRouter);
50 app.listen(port, () => {
51   console.log(`Server running at http://localhost:${port}`);
52 });

```

Screenshots of Output:



Conclusion: We built an API for an online store that can show all products, find specific products, and filter products by category. This teaches us how online stores organize their product information.

Practical 9

Project: Order Management System

Objective: Create a secure API for managing orders with robust logging, error handling, authentication, and session management.

Tasks:

1. CRUD Operations for Orders

- Implement endpoints:
- POST /orders: Create a new order.
- GET /orders: Retrieve all orders.
- GET /orders/:id: Retrieve a specific order by ID.
- PUT /orders/:id: Update an order by ID.
- DELETE /orders/:id: Delete an order by ID.
- Use in-memory storage or a simple database (e.g., MongoDB with Mongoose) for storing orders.

2. Logging Middleware

- Write a custom middleware function to log:
- HTTP method, URL, and timestamp of each API call.
- Save the logs to a file (server.log) using the fs module.

3. Centralized Error Handling

- Implement a centralized error-handling middleware for:
- Invalid endpoints (404 errors).
- Missing or incorrect data in requests (400 errors).
- Server errors (500 errors).
- Use custom error classes for cleaner error handling.

4. Authentication with JWT

- Add user authentication with login and registration endpoints:
- POST /auth/register: Register a new user.
- POST /auth/login: Authenticate a user and return a JWT token.
- Secure order endpoints so only authenticated users can access them.
- Implement middleware to validate JWT tokens in API requests.

5. Session Management

- Use JWT tokens stored in HTTP-only cookies for session management.
- Set the cookie on login and remove it on logout:
- POST /auth/logout: Clear the authentication cookie.

6. Bonus Task: Role-Based Access Control

- Assign roles (e.g., admin, user) during user registration.
- Restrict certain endpoints (e.g., DELETE /orders/:id) to admin users.

Screenshots of Code:

```

});
const orderRouter = express.Router();
orderRouter.post('/', authenticate, (req, res) => {
  const { item, quantity } = req.body;
  if (!item || !quantity) {
    return res.status(400).json({ error: 'Missing required fields' });
  }
  const order = { id: orders.length + 1, item, quantity, user: req.user.username };
  orders.push(order);
  res.status(201).json(order);
});
orderRouter.get('/', authenticate, (req, res) => {
  res.json(orders);
});
orderRouter.get('/:id', authenticate, (req, res) => {
  const order = orders.find(o => o.id === parseInt(req.params.id));
  if (!order) {
    return res.status(404).json({ error: 'Order not found' });
  }
  res.json(order);
});
orderRouter.put('/:id', authenticate, (req, res) => {
  const order = orders.find(o => o.id === parseInt(req.params.id));
  if (!order) {
    return res.status(404).json({ error: 'Order not found' });
  }
  const { item, quantity } = req.body;
  if (item) order.item = item;
  if (quantity) order.quantity = quantity;
  res.json(order);
});
orderRouter.delete('/:id', authenticate, authorizeAdmin, (req, res) => {
  const index = orders.findIndex(o => o.id === parseInt(req.params.id));
  if (index === -1) {
    return res.status(404).json({ error: 'Order not found' });
  }
  orders.splice(index, 1);
  res.status(204).send();
});

```

```

const express = require('express');
const fs = require('fs');
const path = require('path');
const jwt = require('jsonwebtoken');
const cookieParser = require('cookie-parser');
const app = express();
const port = 3000;

app.use(express.json());
app.use(cookieParser());

const SECRET_KEY = 'your_secret_key';
const orders = [];
const users = [];

app.use((req, res, next) => {
  const logEntry = `${new Date().toISOString()} - ${req.method} ${req.url}\n`;
  fs.appendFile('server.log', logEntry, (err) => {
    if (err) {
      console.error('Error writing to log file:', err);
    }
  });
  next();
});

const authenticate = (req, res, next) => {
  const token = req.cookies.token;
  if (!token) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(403).json({ error: 'Invalid token' });
  }
};

```

```
app.use('/orders', orderRouter);
app.post('/auth/register', (req, res) => {
  const { username, password, role } = req.body;
  if (!username || !password || !role) {
    return res.status(400).json({ error: 'Missing required fields' });
  }
  users.push({ username, password, role });
  res.status(201).json({ message: 'User registered' });
});
app.post('/auth/login', (req, res) => {
  const { username, password } = req.body;
  const user = users.find(u => u.username === username && u.password === password);
  if (!user) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  const token = jwt.sign({ username: user.username, role: user.role }, SECRET_KEY, { expiresIn: '1h' });
  res.cookie('token', token, { httpOnly: true }).json({ message: 'Logged in' });
});
app.post('/auth/logout', (req, res) => {
  res.clearCookie('token').json({ message: 'Logged out' });
});
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal Server Error' });
});
app.use((req, res) => {
  res.status(404).json({ error: 'Endpoint not found' });
});
app.listen(port, () => {
  console.log(`Server running at http://localhost:${port}`);
});
```

Screenshots of Output:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1..."
}
```

```
{
  "id": 1,
  "item": "Laptop",
  "quantity": 1,
  "user": "john_doe",
  "status": "Pending"
}
```

```
[
  {
    "id": 1,
    "item": "Laptop",
    "quantity": 1,
    "user": "john_doe",
    "status": "Pending"
  }
]
```

```
{
  "id": 1,
  "item": "Laptop",
  "quantity": 1,
  "user": "john_doe",
  "status": "Shipped"
}
```

```
{
  "message": "Order deleted successfully"
}
```

Conclusion: This practical taught us how to create a secure system for managing orders, with user login, error handling, and access control. This helps understand how online shops keep track of orders and ensure only authorized people can see them.

Practical 10

Project: Personal Task Manager

Objective: Build a task management API for creating, updating, and managing tasks.

Tasks:

- CRUD operations for tasks (POST, GET, PUT, DELETE).
- Validate task input data (e.g., title and status fields) using middleware.
- Persist tasks in a tasks.json file.

Screenshots of Code:

```
1  const express = require('express');
2  const fs = require('fs');
3  const path = require('path');
4  const app = express();
5  const port = 3000;
6
7  app.use(express.json());
8  app.use(express.urlencoded({ extended: true }));
9
10 const tasksFile = 'tasks.json';
11
12 // Load tasks from file
13 const loadTasks = () => {
14   if (!fs.existsSync(tasksFile)) return [];
15   const data = fs.readFileSync(tasksFile);
16   return JSON.parse(data);
17 };
18
19 // Save tasks to file
20 const saveTasks = (tasks) => {
21   fs.writeFileSync(tasksFile, JSON.stringify(tasks, null, 2));
22 };
23
24 let tasks = loadTasks();
25
26 // Middleware to validate task input
27 const validateTask = (req, res, next) => {
28   const { title, status } = req.body;
29   if (!title || typeof title !== 'string') {
30     return res.status(400).json({ error: 'Title is required and must be a string' });
31   }
```

```
31     }
32     if (status && ![ 'pending', 'completed' ].includes(status)) {
33         return res.status(400).json({ error: 'Invalid status value' });
34     }
35     next();
36 };
37
38 // CRUD Endpoints for Tasks
39 app.post('/tasks', validateTask, (req, res) => {
40     const { title, status = 'pending' } = req.body;
41     const task = { id: tasks.length + 1, title, status };
42     tasks.push(task);
43     saveTasks(tasks);
44     res.status(201).json(task);
45 });
46
47 app.get('/tasks', (req, res) => {
48     res.json(tasks);
49 });
50
51 app.get('/tasks/:id', (req, res) => {
52     const task = tasks.find(t => t.id === parseInt(req.params.id));
53     if (!task) {
54         return res.status(404).json({ error: 'Task not found' });
55     }
56     res.json(task);
57 });
58
59 app.put('/tasks/:id', validateTask, (req, res) => {
```

```
59 app.put('/tasks/:id', validateTask, (req, res) => {
60   const task = tasks.find(t => t.id === parseInt(req.params.id));
61   if (!task) {
62     return res.status(404).json({ error: 'Task not found' });
63   }
64   const { title, status } = req.body;
65   if (title) task.title = title;
66   if (status) task.status = status;
67   saveTasks(tasks);
68   res.json(task);
69 });
70
71 app.delete('/tasks/:id', (req, res) => {
72   const index = tasks.findIndex(t => t.id === parseInt(req.params.id));
73   if (index === -1) {
74     return res.status(404).json({ error: 'Task not found' });
75   }
76   tasks.splice(index, 1);
77   saveTasks(tasks);
78   res.status(204).send();
79 });
80
81 // 404 Handler
82 app.use((req, res) => {
83   res.status(404).json({ error: 'Endpoint not found' });
84 });
85
86 app.listen(port, () => {
87   console.log(`Server running at http://localhost:\${port}`);
88 });
```

Screenshots of Output:

```
{  
  "title": "Learn Node.js",  
  "status": "Completed",  
  "id": 1  
}
```

Task deleted

Conclusion: We built an API that helps manage personal tasks, allowing us to create, view, update, and delete tasks. This teaches us how task management apps work behind the scenes.

Practical 11

Project: User Profile Manager

Objective: Set up MongoDB and Mongoose in a Node.js application and create a basic schema to perform simple database operations.

Tasks:

- Install MongoDB locally or create a free cluster on MongoDB Atlas.
- Set up a new Node.js project and install the required dependencies (express, mongoose, dotenv).
- Create a .env file and store your MongoDB connection URI securely.
- Write a connection script in Node.js using Mongoose to connect to MongoDB.
- Define a user schema with fields like name, email, and age.
- Create a simple script to insert a user into the database and log the results in the console.
- Fetch all users from the database and display them in the console.

Screenshots of Code:

Server.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const User = require('./models/User');

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Error connecting to MongoDB:', err));

app.post('/users', async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();
    res.status(201).send(user);
  } catch (error) {
    res.status(400).send(error);
  }
});

app.get('/users', async (req, res) => {
  try {
    const users = await User.find();
    res.send(users);
  } catch (error) {
    res.status(500).send(error);
  }
});

app.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

Script/fetchusers.js

```
require('dotenv').config();
const mongoose = require('mongoose');
const User = require('../models/User');

mongoose.connect(process.env.MONGODB_URI);

User.find()
  .then(users => {
    console.log('All users:', users);
    mongoose.connection.close();
  })
  .catch(err => {
    console.error('Error fetching users:', err);
    mongoose.connection.close();
  });
```

Script/insertusers.js

```
require('dotenv').config();
const mongoose = require('mongoose');
const User = require('../models/User');
const readline = require('readline').createInterface({
  input: process.stdin,
  output: process.stdout
});

mongoose.connect(process.env.MONGODB_URI);

async function createUser() {
  readline.question('Enter user name: ', async (name) => {
    readline.question('Enter user email: ', async (email) => {
      readline.question('Enter user age: ', async (age) => {
        const newUser = new User({
          name,
          email,
          age: parseInt(age)
        });

        try {
          const user = await newUser.save();
          console.log('User created:', user);
        } catch (err) {
          console.error('Error creating user:', err);
        } finally {
          mongoose.connection.close();
          readline.close();
        }
      });
    });
  });
}

createUser();
```

models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  age: {
    type: Number,
    required: true
  }
});

module.exports = mongoose.model('User', userSchema);
```

.env

```
MONGODB_URI=mongodb://localhost:27017/userProfileManager
PORT=3002
```

Screenshots of Output:

```
PS D:\fswd\practical11> node server.js
Server running on port 3002
Connected to MongoDB
```

```
PS D:\fswd\practical11> node scripts/fetchUsers.js
All users: [
  {
    _id: new ObjectId('67e9155ecbe90e14883caefa'),
    name: 'John Doe',
    email: 'john@example.com',
    age: 30,
    __v: 0
  },
  {
    _id: new ObjectId('67e9155ecbe90e14883caefa'),
    name: 'John Doe',
    email: 'john@example.com',
    age: 30,
    __v: 0
  }
]
```

Conclusion: This practical showed us how to connect to a database and store user information. We learned how to add users to the database and retrieve them later. This helps understand how websites store user data.

Practical 12

Project: Task Manager API

Objective: Build a RESTful API with Express and Mongoose to manage tasks in a MongoDB collection.

Tasks:

- Define a Task schema with fields like title, description, status (e.g., "Pending", "Completed"), and dueDate.
- Set up the following API endpoints:
 - POST /tasks: Add a new task to the database.
 - GET /tasks: Retrieve all tasks.
 - GET /tasks/:id: Retrieve a specific task by ID.
 - PUT /tasks/:id: Update task details by ID.
 - DELETE /tasks/:id: Delete a task by ID.
- Test the API endpoints using Postman or Thunder Client.
- Use filters to query tasks based on their status or dueDate.
- Handle basic errors, such as invalid task IDs or missing fields.

Screenshots of Code:

.env

```
MONGODB_URI=mongodb://localhost:27017/taskManager
PORT=3000
```

model/task.js

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    trim: true
  },
  status: {
    type: String,
    enum: ['Pending', 'Completed'],
    default: 'Pending'
  },
  dueDate: {
    type: Date,
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model('Task', taskSchema);
```

server.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const Task = require('./model/Task');

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Error connecting to MongoDB:', err));

app.post('/tasks', async (req, res) => {
  try {
    const task = new Task(req.body);
    await task.save();
    res.status(201).send(task);
  } catch (error) {
    res.status(400).send(error);
  }
});

app.get('/tasks', async (req, res) => {
  try {
    const { status, dueDate } = req.query;
    const filter = {};
    if (status) filter.status = status;
    if (dueDate) filter.dueDate = { $lte: new Date(dueDate) };
    const tasks = await Task.find(filter);
    res.send(tasks);
  } catch (error) {
    res.status(500).send(error);
  }
});

app.get('/tasks/:id', async (req, res) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) return res.status(404).send('Task not found');
    res.send(task);
  } catch (error) {
```

```
        res.status(500).send(error);
    }
});

app.put('/tasks/:id', async (req, res) => {
    try {
        const task = await Task.findByIdAndUpdate(req.params.id, req.body, { new: true });
        if (!task) return res.status(404).send('Task not found');
        res.send(task);
    } catch (error) {
        res.status(400).send(error);
    }
});

app.delete('/tasks/:id', async (req, res) => {
    try {
        const task = await Task.findByIdAndDelete(req.params.id);
        if (!task) return res.status(404).send('Task not found');
        res.send(task);
    } catch (error) {
        res.status(500).send(error);
    }
});

app.listen(process.env.PORT, () => {
    console.log(`Server running on port ${process.env.PORT}`);
});
```

Screenshots of Output:

POST ☐ http://localhost:3000/tasks

Query Headers 3 Auth **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "title": "Term work submission",
3   "description": "Finish practicals",
4   "status": "Pending",
5   "dueDate": "2025-04-04"
6 }
```

Status: 201 Created Size: 241 Bytes Time: 29 ms

Response Headers 6 Cookies Results Docs {}

```
1 {
2   "title": "Term work submission",
3   "description": "Finish practicals",
4   "status": "Pending",
5   "dueDate": "2025-04-04T00:00:00.000Z",
6   "_id": "67e93266729f86fcb82672c",
7   "createdAt": "2025-03-30T12:00:38.660Z",
8   "updatedAt": "2025-03-30T12:00:38.660Z",
9   "__v": 0
10 }
```

GET ☐ http://localhost:3000/tasks

Query Headers 3 Auth **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "title": "Term work submission",
3   "description": "Finish practicals",
4   "status": "Pending",
5   "dueDate": "2025-04-04"
6 }
```

Status: 200 OK Size: 969 Bytes Time: 13 ms

Response Headers 6 Cookies Results Docs {}

```
12 {
13   "_id": "67e928187683a269b40b43f0",
14   "title": "learn fswd",
15   "description": "Finish the Task of FSWD",
16   "status": "Pending",
17   "dueDate": "2025-03-31T00:00:00.000Z",
18   "createdAt": "2025-03-30T11:16:40.811Z",
19   "updatedAt": "2025-03-30T11:16:40.811Z",
20   "__v": 0
21 },
22 {
23   "_id": "67e928547683a269b40b43f3",
24   "title": "learn fswd",
25   "description": "Finish the Task of FSWD",
26   "status": "Pending",
27   "dueDate": "2025-03-31T00:00:00.000Z",
28   "createdAt": "2025-03-30T11:17:40.506Z",
29   "updatedAt": "2025-03-30T11:17:40.506Z",
30   "__v": 0
31 },
32 {
33   "_id": "67e93266729f86fcb82672c",
34   "title": "Term work submission",
35   "description": "Finish practicals",
36   "status": "Pending",
37   "dueDate": "2025-04-04T00:00:00.000Z",
38   "createdAt": "2025-03-30T12:00:38.660Z",
39   "updatedAt": "2025-03-30T12:00:38.660Z",
40   "__v": 0
41 }
```

PUT ☐ http://localhost:3000/tasks/67e928547683a269b40b43f3

Query Headers 3 Auth **Body 1** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "status": "Completed"
3 }
```

Status: 200 OK Size: 239 Bytes Time: 20 ms

Response Headers 6 Cookies Results Docs {}

```
1 {
2   "_id": "67e928547683a269b40b43f3",
3   "title": "learn fswd",
4   "description": "Finish the Task of FSWD",
5   "status": "Completed",
6   "dueDate": "2025-03-31T00:00:00.000Z",
7   "createdAt": "2025-03-30T11:17:40.506Z",
8   "updatedAt": "2025-03-30T12:03:01.410Z",
9   "__v": 0
10 }
```

GET Send

Status: 200 OK Size: 481 Bytes Time: 15 ms

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```

1 {
2   "title": "Term work submission",
3   "description": "Finish practicals",
4   "status": "Pending",
5   "dueDate": "2025-04-04"
6 }

```

Response Headers 6 Cookies Results Docs {}

```

1 [
2   {
3     "_id": "67e928187683a269b40b43f0",
4     "title": "learn fswd",
5     "description": "Finish the Task of FSWD",
6     "status": "Pending",
7     "dueDate": "2025-03-31T00:00:00.000Z",
8     "createdAt": "2025-03-30T11:16:40.811Z",
9     "updatedAt": "2025-03-30T11:16:40.811Z",
10    "__v": 0
11  },
12  {
13    "_id": "67e93266729f86fcb82672c",
14    "title": "Term work submission",
15    "description": "Finish practicals",
16    "status": "Pending",
17    "dueDate": "2025-04-04T00:00:00.000Z",
18    "createdAt": "2025-03-30T12:00:38.660Z",
19    "updatedAt": "2025-03-30T12:00:38.660Z",
20    "__v": 0
21  }
22 ]

```

DELETE Send

Status: 200 OK Size: 237 Bytes Time: 17 ms

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```

1 {
2   "title": "Term work submission",
3   "description": "Finish practicals",
4   "status": "Pending",
5   "dueDate": "2025-04-04"
6 }

```

Response Headers 6 Cookies Results Docs {}

```

1 {
2   "_id": "67e928187683a269b40b43f0",
3   "title": "learn fswd",
4   "description": "Finish the Task of FSWD",
5   "status": "Pending",
6   "dueDate": "2025-03-31T00:00:00.000Z",
7   "createdAt": "2025-03-30T11:16:40.811Z",
8   "updatedAt": "2025-03-30T11:16:40.811Z",
9   "__v": 0
10 }

```

Conclusion: We built a complete system for managing tasks using a database. We can add, view, update, and delete tasks, and even search for specific tasks. This teaches us how task management apps store and organize information.

Practical 13

Project: Blog Application API

Objective: Implement a relational data model to create and manage blog posts with associated authors using Mongoose and Express.

Tasks:

- Define schemas for Author and BlogPost:
 - Author: name, email.
 - BlogPost: title, content, createdAt, author (reference to Author).
- Set up the following API endpoints:
 - POST /authors: Add a new author.
 - POST /blogposts: Add a new blog post and associate it with an author.
 - GET /blogposts: Retrieve all blog posts, including author details (use populate).
 - GET /blogposts/:id: Retrieve a specific blog post by ID with author details.
 - DELETE /blogposts/:id: Delete a blog post by ID.
- Add Mongoose validation to ensure all required fields are provided and correctly formatted (e.g., email validation).
- Handle errors gracefully, such as missing or invalid IDs and validation failures.

Screenshots of Code:

.env

```
MONGODB_URI=mongodb://localhost:27017/blogDB
PORT=3001
```

server.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const authorRoutes = require('./routes/authorRoutes');
const blogPostRoutes = require('./routes/blogPostRoutes');

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Error connecting to MongoDB:', err));

app.use('/authors', authorRoutes);
app.use('/blogposts', blogPostRoutes);

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
});

app.listen(process.env.PORT, () => {
  console.log(`Server running on port ${process.env.PORT}`);
});
```

models/Author.js

```
const mongoose = require('mongoose');

const authorSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Author name is required'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    trim: true,
    lowercase: true,
    validate: {
      validator: (value) => /^\\S+@\\S+\\.\\S+$/\\.test(value),
      message: 'Invalid email format'
    }
  }
});

module.exports = mongoose.model('Author', authorSchema);
```

Models/BlogPost.js

```
const mongoose = require('mongoose');

const blogPostSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Title is required'],
    trim: true
  },
  content: {
    type: String,
    required: [true, 'Content is required']
  },
  createdAt: {
    type: Date,
    default: Date.now
  },
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Author',
    required: [true, 'Author reference is required']
  }
});

module.exports = mongoose.model('BlogPost', blogPostSchema);
```

routes/authorRoutes.js

```
const express = require('express');
const { createAuthor } = require('../controllers/authorController');

const router = express.Router();

router.post('/', createAuthor);

module.exports = router;
```

routes/blogpostRoutes.js

```
const router = express.Router();

router.post('/', createBlogPost);
router.get('/', getAllBlogPosts);
router.get('/:id', getBlogPostById);
router.delete('/:id', deleteBlogPost);

module.exports = router;
```

controllers/authorController.js

```
const Author = require('../models/Author');

exports.createAuthor = async (req, res) => {
  try {
    const author = new Author(req.body);
    await author.save();
    res.status(201).json(author);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};
```

Controllers/blogpostController.js

```
const BlogPost = require('../models/BlogPost');
const Author = require('../models/Author');

exports.createBlogPost = async (req, res) => {
  try {
    const authorExists = await Author.findById(req.body.author);
    if (!authorExists) {
      return res.status(404).json({ error: 'Author not found' });
    }
    const blogPost = new BlogPost(req.body);
    await blogPost.save();
    res.status(201).json(blogPost);
  } catch (error) {
    res.status(400).json({ error: error.message });
  }
};

exports.getAllBlogPosts = async (req, res) => {
  try {
    const blogPosts = await BlogPost.find().populate('author', 'name email');
    res.json(blogPosts);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

exports.getBlogPostById = async (req, res) => {
  try {
    const blogPost = await BlogPost.findById(req.params.id).populate('author', 'name email');
    if (!blogPost) {
      return res.status(404).json({ error: 'Blog post not found' });
    }
    res.json(blogPost);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

exports.deleteBlogPost = async (req, res) => {
  try {
    const blogPost = await BlogPost.findByIdAndDelete(req.params.id);
    if (!blogPost) {
      return res.status(404).json({ error: 'Blog post not found' });
    }
    res.json({ message: 'Blog post deleted successfully' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

Screenshots of Output:

```
PS D:\fswd\practical13> node server.js
Server running on port 3001
Connected to MongoDB
```

POST

Query Headers ³ Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 { "name": "Collen Hoover", "email": "collen@gmail.com" }
```

Status: 201 Created Size: 92 Bytes Time: 14 ms

Response Headers ⁶ Cookies Results Docs

```
1 {
2   "name": "Collen Hoover",
3   "email": "collen@gmail.com",
4   "_id": "67e935de14b256ddaea33c02",
5   "__v": 0
6 }
```

POST

Query Headers ³ Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 { "title": "Verity", "content": "Thriller", "author":
   "67e935de14b256ddaea33c02" }
```

Status: 201 Created Size: 155 Bytes Time: 21 ms

Response Headers ⁶ Cookies Results Docs

```
1 {
2   "title": "Verity",
3   "content": "Thriller",
4   "author": "67e935de14b256ddaea33c02",
5   "_id": "67e9366714b256ddaea33c05",
6   "createdAt": "2025-03-30T12:17:43.358Z",
7   "__v": 0
8 }
```

GET

Query Headers ³ Auth **Body ¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 { "title": "Verity", "content": "Thriller", "author":
   "67e935de14b256ddaea33c02" }
```

Status: 200 OK Size: 438 Bytes Time: 30 ms

Response Headers ⁶ Cookies Results Docs

```
1 [
2   {
3     "_id": "67e92c87726fa92a84019d32",
4     "title": "First Part",
5     "content": "Hello World",
6     "author": {
7       "_id": "67e92c2b726fa92a84019d2f",
8       "name": "Adeline Reilly",
9       "email": "adeline@gmail.com"
10    },
11    "createdAt": "2025-03-30T11:35:35.499Z",
12    "__v": 0
13  },
14  {
15    "_id": "67e9366714b256ddaea33c05",
16    "title": "Verity",
17    "content": "Thriller",
18    "author": {
19      "_id": "67e935de14b256ddaea33c02",
20      "name": "Collen Hoover",
21      "email": "collen@gmail.com"
22    },
23    "createdAt": "2025-03-30T12:17:43.358Z",
24    "__v": 0
25  }
26 ]
```

The image displays two screenshots of a REST client interface, likely Postman, showing the results of two API requests.

Top Screenshot (GET Request):

- Method:** GET
- URL:** `http://localhost:3001/blogposts/67e9366714b256ddaea33c05`
- Status:** 200 OK
- Size:** 213 Bytes
- Time:** 10 ms
- Body:** The request body is in JSON format, containing a single object with the following fields: `{ "title": "Verity", "content": "Thriller", "author": "67e935de14b256ddaea33c02" }`.
- Response:** The response is in JSON format, containing a single object with the following fields: `{ "_id": "67e9366714b256ddaea33c05", "title": "Verity", "content": "Thriller", "author": { "_id": "67e935de14b256ddaea33c02", "name": "Collen Hoover", "email": "collen@gmail.com" }, "createdAt": "2025-03-30T12:17:43.358Z", "__v": 0 }`.

Bottom Screenshot (DELETE Request):

- Method:** DELETE
- URL:** `http://localhost:3001/blogposts/67e9366714b256ddaea33c05`
- Status:** 200 OK
- Size:** 44 Bytes
- Time:** 6 ms
- Body:** The request body is in JSON format, containing a single object with the following fields: `{ "title": "Verity", "content": "Thriller", "author": "67e935de14b256ddaea33c02" }`.
- Response:** The response is in JSON format, containing a single object with the following field: `{ "message": "Blog post deleted successfully" }`.

Conclusion: This practical taught us how to create a blog system where authors can write posts. We learned how to connect authors to their blog posts in a database. This helps understand how blogs organize content and authors.

Practical 14

Project: Personal Profile Card

Objective: Learn the basics of React by building reusable components and passing data using props.

Tasks:

- Set up a React project using Vite.
- Create a functional component for a profile card.
- Use props to display a user's name, photo, and a short bio.
- Style the card using basic CSS.

Screenshots of Code:

```

.profile-card {
  width: 300px;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.2);
  text-align: center;
  font-family: Arial, sans-serif;
  background-color: #f9f9f9;
}

.profile-photo {
  width: 100px;
  height: 100px;
  border-radius: 50%;
  object-fit: cover;
  margin-bottom: 10px;
}

h2 {
  margin: 0;
  font-size: 20px;
}

p {
  color: #666;
}

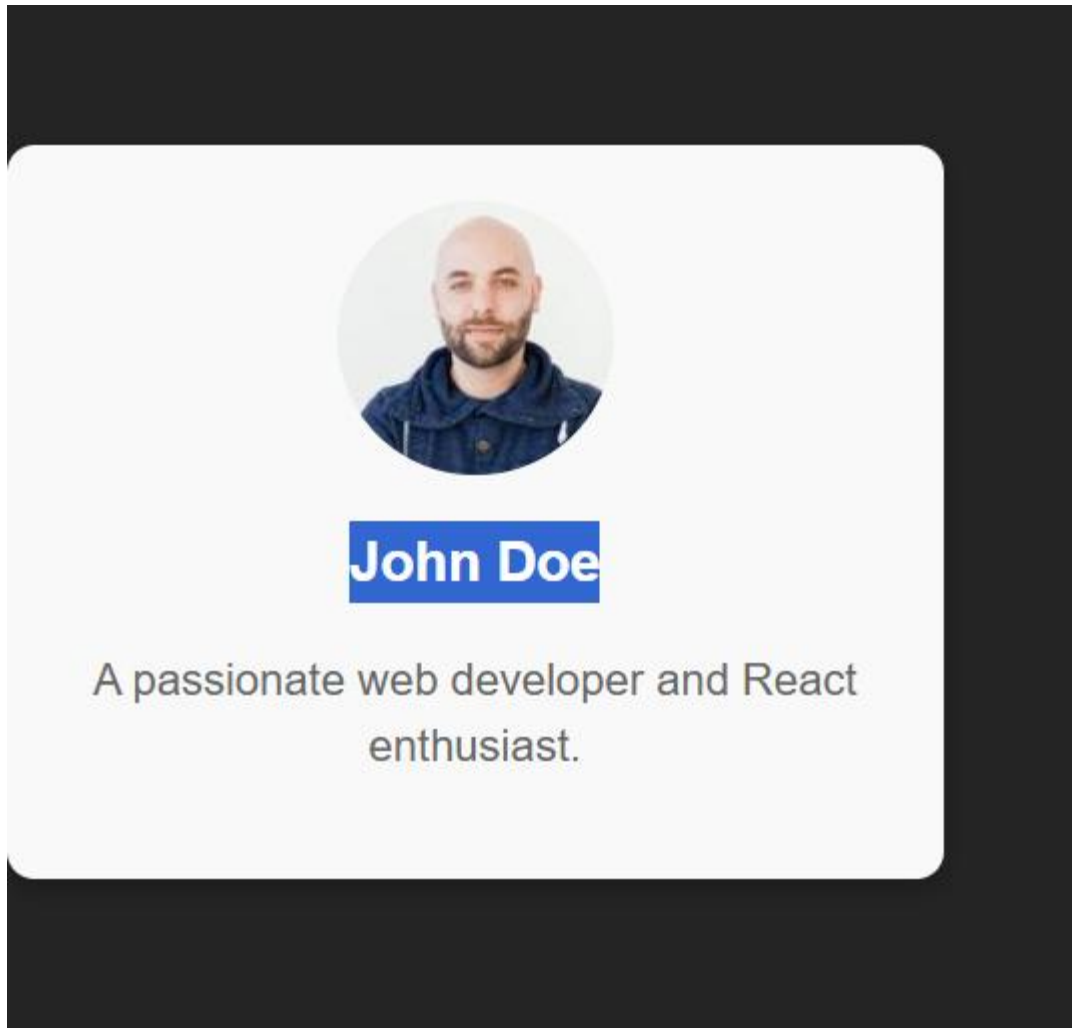
```

```

1  import React from "react";
2  import "../ProfileCard.css"; // Import CSS file for styling
3
4  const ProfileCard = ({ name, photo, bio }) => {
5    return (
6      <div className="profile-card">
7        <img src={photo} alt={name} className="profile-photo" />
8        <h2>{name}</h2>
9        <p>{bio}</p>
10     </div>
11   );
12 };
13
14 export default ProfileCard;
15

```

```
1  import React from "react";
2  import ProfileCard from "../components/ProfileCard";
3
4  const App = () => {
5    return (
6      <div style={{ display: "flex", justifyContent: "center", marginTop: "50px" }}>
7        <ProfileCard
8          name="John Doe"
9          photo="https://randomuser.me/api/portraits/men/75.jpg"
10         bio="A passionate web developer and React enthusiast."
11        />
12      </div>
13    );
14  };
15
16  export default App;
17
```

Screenshots of Output :

Conclusion: We created a simple profile card using React that shows a person's name, photo, and bio. This teaches us the basics of building user interfaces with React components.

Practical 15

Project: Simple Counter App

Objective: Understand how to manage and update state dynamically in a React application.

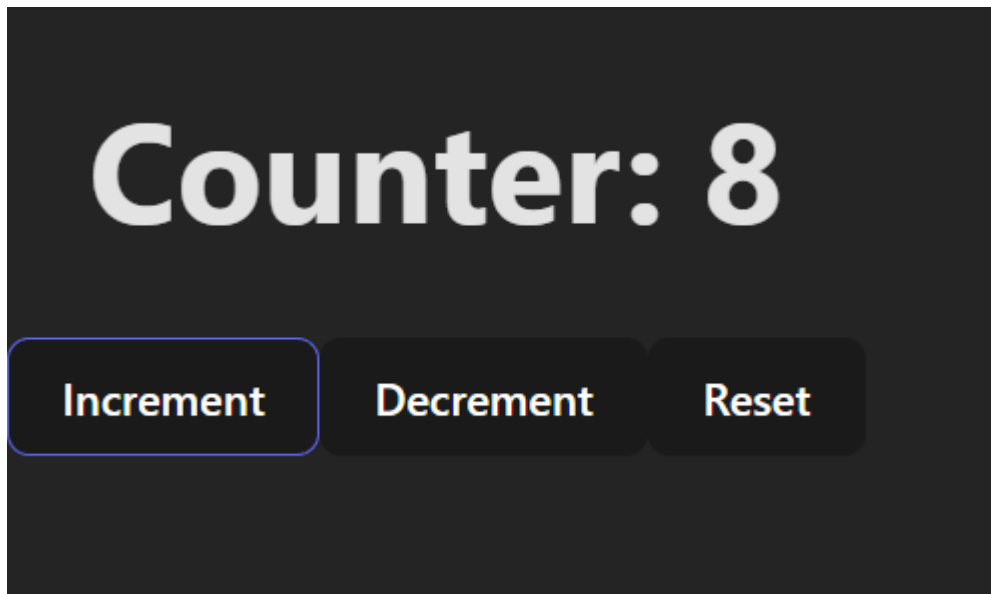
Tasks:

- Create a functional component with a state variable for the counter.
- Add buttons to increment, decrement, and reset the counter.
- Display the counter value dynamically on the screen.

Screenshots of Code:

```
1  import { useState } from "react";
2
3  const Counter = () => {
4    const [count, setCount] = useState(0);
5
6    return (
7      <div style={{ textAlign: "center", marginTop: "50px" }}>
8        <h1>Counter: {count}</h1>
9        <button onClick={() => setCount(count + 1)}>Increment</button>
10       <button onClick={() => setCount(count - 1)}>Decrement</button>
11       <button onClick={() => setCount(0)}>Reset</button>
12     </div>
13   );
14 };
15
16 export default Counter;
17
```

```
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import Counter from "./App.jsx";
4  import "./index.css";
5
6  ReactDOM.createRoot(document.getElementById("root")).render(
7    <React.StrictMode>
8      <Counter />
9    </React.StrictMode>
10 );
11
```

Screenshots of Output :

Conclusion: This practical showed us how to create a counter that can be increased, decreased, or reset using React. This helps understand how to make interactive websites that respond to user actions.

Practical 16

Project: To-Do List App

Objective: Learn how to dynamically render and manage a list of items with conditional rendering.

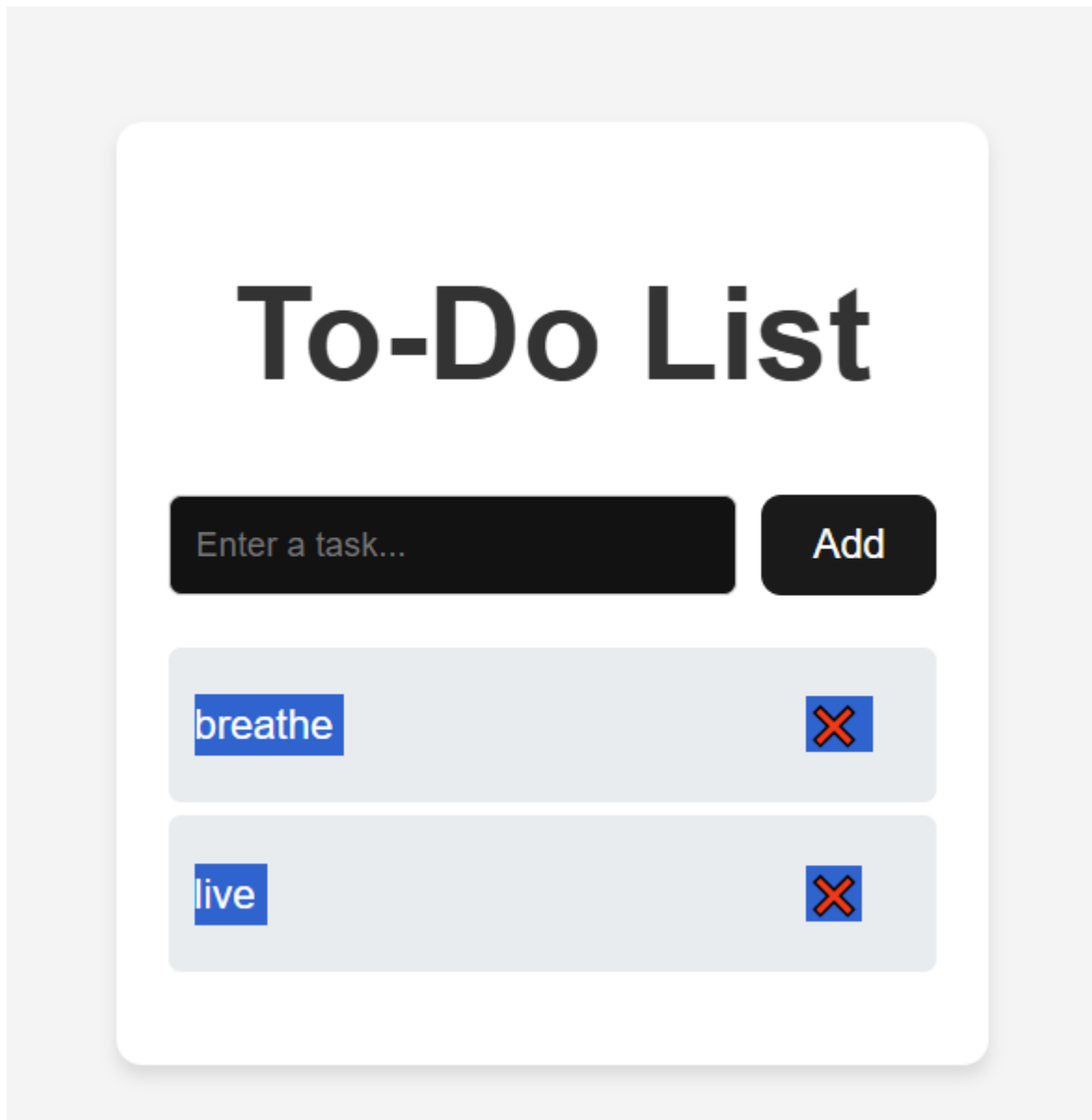
Tasks:

- Create a state variable to store a list of tasks.
- Render the task list dynamically using `.map()`.
- Add functionality to add and remove tasks.
- Display a message when the task list is empty.

Screenshots of Code

```
1  import { useState } from "react";
2  import "./App.css";
3
4  const ToDoList = () => {
5    const [tasks, setTasks] = useState([]);
6    const [newTask, setNewTask] = useState("");
7
8    const addTask = () => {
9      if (newTask.trim() === "") return;
10     setTasks([...tasks, newTask]);
11     setNewTask("");
12   };
13
14   const removeTask = (index) => {
15     const updatedTasks = tasks.filter((_, i) => i !== index);
16     setTasks(updatedTasks);
17   };
18
19   return (
20     <div className="todo-container">
21       <h1>To-Do List</h1>
22       <div className="input-container">
23         <input
24           type="text"
25           placeholder="Enter a task..."
26           value={newTask}
27           onChange={(e) => setNewTask(e.target.value)}
28         />
29         <button onClick={addTask}>Add</button>
30       </div>
31     </div>
32   );
33 }
```

```
23     <input
24       type="text"
25       placeholder="Enter a task..."
26       value={newTask}
27       onChange={(e) => setNewTask(e.target.value)}
28     />
29     <button onClick={addTask}>Add</button>
30   </div>
31   {tasks.length === 0 ? (
32     <p className="empty-message">No tasks yet! Add one above.</p>
33   ) : (
34     <ul>
35       {tasks.map((task, index) => (
36         <li key={index}>
37           {task}
38           <button className="delete-btn" onClick={() => removeTask(index)}>✕</but
39         </li>
40       ))}
41     </ul>
42   )}
43 </div>
44 );
45 };
46
47 export default ToDoList;
48 |
```

Screenshots of Output :

Conclusion: We built a to-do list app where users can add and remove tasks. This teaches us how to create and manage lists of information in React applications.

Practical 17

Project: Feedback Form

Objective: Understand how to handle and validate user input in a React form.

Tasks:

- Create a form with input fields for name, email, and feedback message.
- Use state variables to control the form inputs.
- Validate the inputs (e.g., ensure fields are not empty).
- Display submitted feedback data below the form.

Screenshots of Code:

```

48      <input type="email" name="email" value={formData.email} onChange={handleChange} /
49      {errors.email && <p className="error">{errors.email}</p>}
50    </div>
51
52    <div className="form-group">
53      <label>Feedback:</label>
54      <textarea name="message" value={formData.message} onChange={handleChange}></textare
55      {errors.message && <p className="error">{errors.message}</p>}
56    </div>
57
58    <button type="submit">Submit</button>
59  </form>
60
61  {submittedFeedback.length > 0 && (
62    <div className="feedback-list">
63      <h2>Submitted Feedback</h2>
64      {submittedFeedback.map((feedback, index) => (
65        <div key={index} className="feedback-card">
66          <h3>{feedback.name}</h3>
67          <p><strong>Email:</strong> {feedback.email}</p>
68          <p>{feedback.message}</p>
69        </div>
70      ))}
71    </div>
72  )}
73 </div>
74 );
75 };
76 };
77

```

```

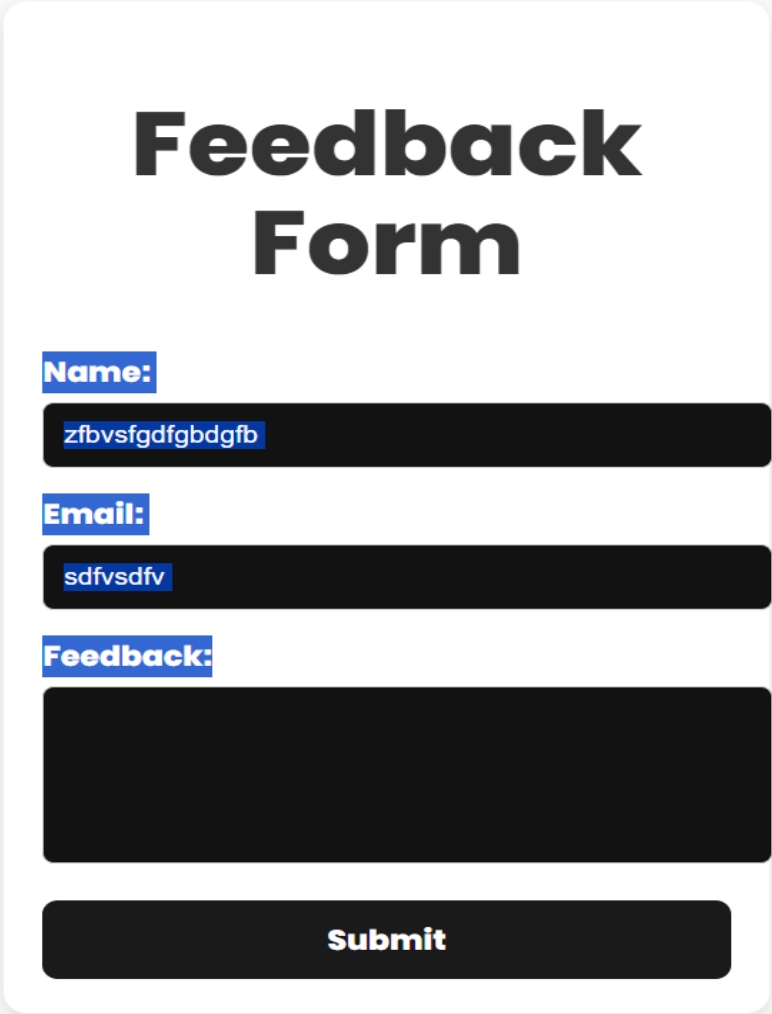
1  import { useState } from "react";
2  import "../App.css";
3
4  const FeedbackForm = () => {
5    const [formData, setFormData] = useState({ name: "", email: "", message: "" });
6    const [submittedFeedback, setSubmittedFeedback] = useState([]);
7    const [errors, setErrors] = useState({});
8
9    const handleChange = (e) => {
10      setFormData({ ...formData, [e.target.name]: e.target.value });
11    };
12
13    const validateForm = () => {
14      let newErrors = {};
15      if (!formData.name.trim()) newErrors.name = "Name is required.";
16      if (!formData.email.trim()) {
17        newErrors.email = "Email is required.";
18      } else if (!/^\S+@\S+\.\S+$/i.test(formData.email)) {
19        newErrors.email = "Enter a valid email.";
20      }
21      if (!formData.message.trim()) newErrors.message = "Feedback cannot be empty.";
22      return newErrors;
23    };
24
25    const handleSubmit = (e) => {
26      e.preventDefault();
27      const validationErrors = validateForm();
28      if (Object.keys(validationErrors).length > 0) {
29        setErrors(validationErrors);
30      } else {

```

```
1  body {
2    font-family: "Poppins", sans-serif;
3    background-color: #f9f9f9;
4    display: flex;
5    justify-content: center;
6    align-items: center;
7    height: 100vh;
8    margin: 0;
9  }
10
11  .container {
12    background: white;
13    padding: 20px;
14    border-radius: 12px;
15    box-shadow: 0px 4px 10px rgba(0, 0, 0, 0.1);
16    width: 350px;
17    text-align: center;
18  }
19
20  h1 {
21    color: #333;
22  }
23
24  .form-group {
25    margin-bottom: 15px;
26    text-align: left;
27  }
28
29  label {
30    font-weight: bold;
31    display: block;
32    margin-bottom: 5px;
33  }
34
35  input, textarea {
36    width: 100%;
37    padding: 10px;
38    border: 1px solid #ccc;
39    border-radius: 6px;
40    font-size: 14px;
41  }
42
43  textarea {
44    height: 80px;
45    resize: none;
46  }
47
48  button {
49    background-color: #007bff;
50    color: white;
51    border: none;
52    padding: 12px;
53    width: 100%;
```

```
1 import React from "react";
2 import ReactDOM from "react-dom/client";
3 import FeedbackForm from "./App.jsx";
4 import "./index.css";
5
6 ReactDOM.createRoot(document.getElementById("root")).render(
7   <React.StrictMode>
8     <FeedbackForm />
9   </React.StrictMode>
10 );
11
```

Screenshots of Output :



Feedback Form

Name:
zfbvsfgdfgbdgfb

Email:
sdfvsdfv

Feedback:

Submit

Conclusion: This practical taught us how to create a form that collects user feedback, checks if the information is correct, and shows the submitted feedback. This helps understand how websites collect and validate user input.

Practical 18

Project: Theme Switcher App

Objective: Learn how to use the Context API to manage global state across components.

Tasks:

- Set up a Context Provider to manage theme state.
- Implement a toggle button to switch between light and dark themes.
- Use the theme context to dynamically update the app's background and text colours.

Screenshots of Code:

```

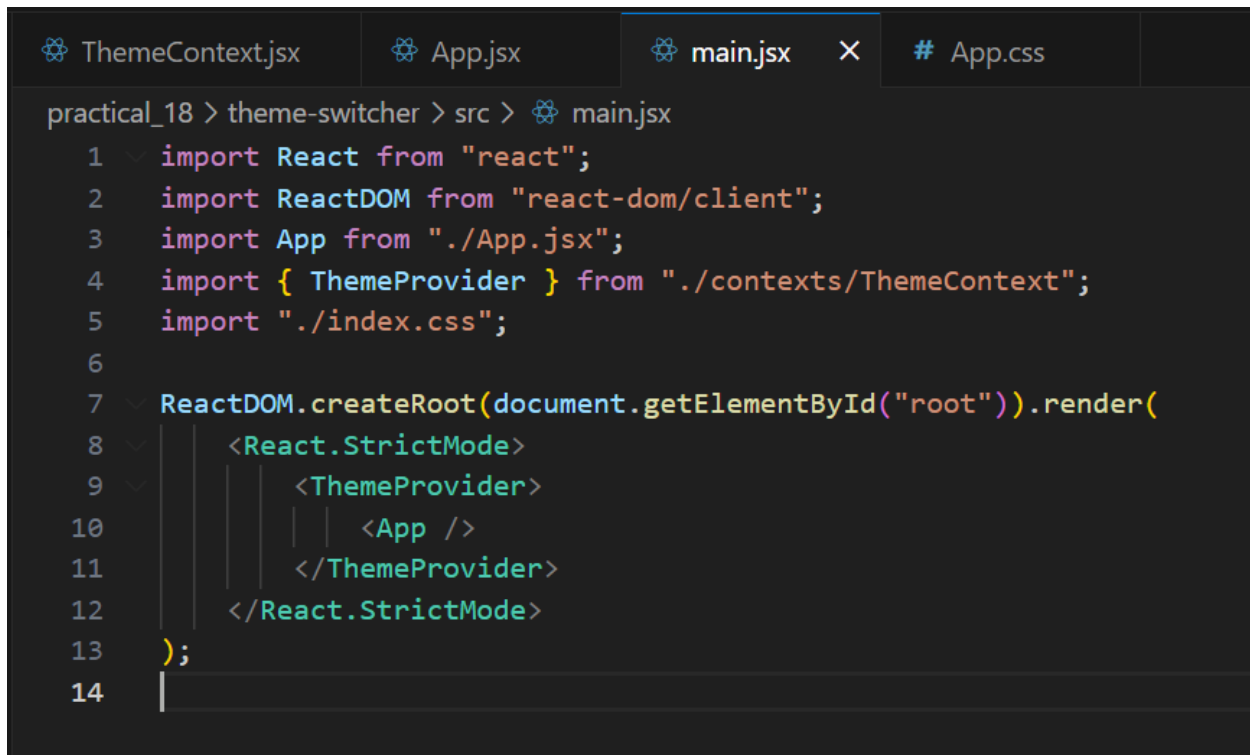
ThemeContext.jsx × App.jsx × main.jsx # App.css
D:\charusat\sem 4\full_stack\practicals\practical_18\theme-switcher\src\contexts\ThemeContext.jsx from "../contexts/ThemeContext";
2 import { FaSun, FaMoon } from "react-icons/fa";
3 import "../App.css";
4
5 const App = () => {
6   const { theme, toggleTheme } = useTheme();
7
8   return (
9     <div className={`app ${theme}`}>
10      <h1>Theme Switcher</h1>
11      <button onClick={toggleTheme} className="toggle-btn">
12        {theme === "light" ? <FaMoon /> : <FaSun />}
13        {theme === "light" ? " Dark Mode" : " Light Mode"}
14      </button>
15      <p>Click the button to switch themes!</p>
16    </div>
17  );
18 };
19
20 export default App;
21

```

```

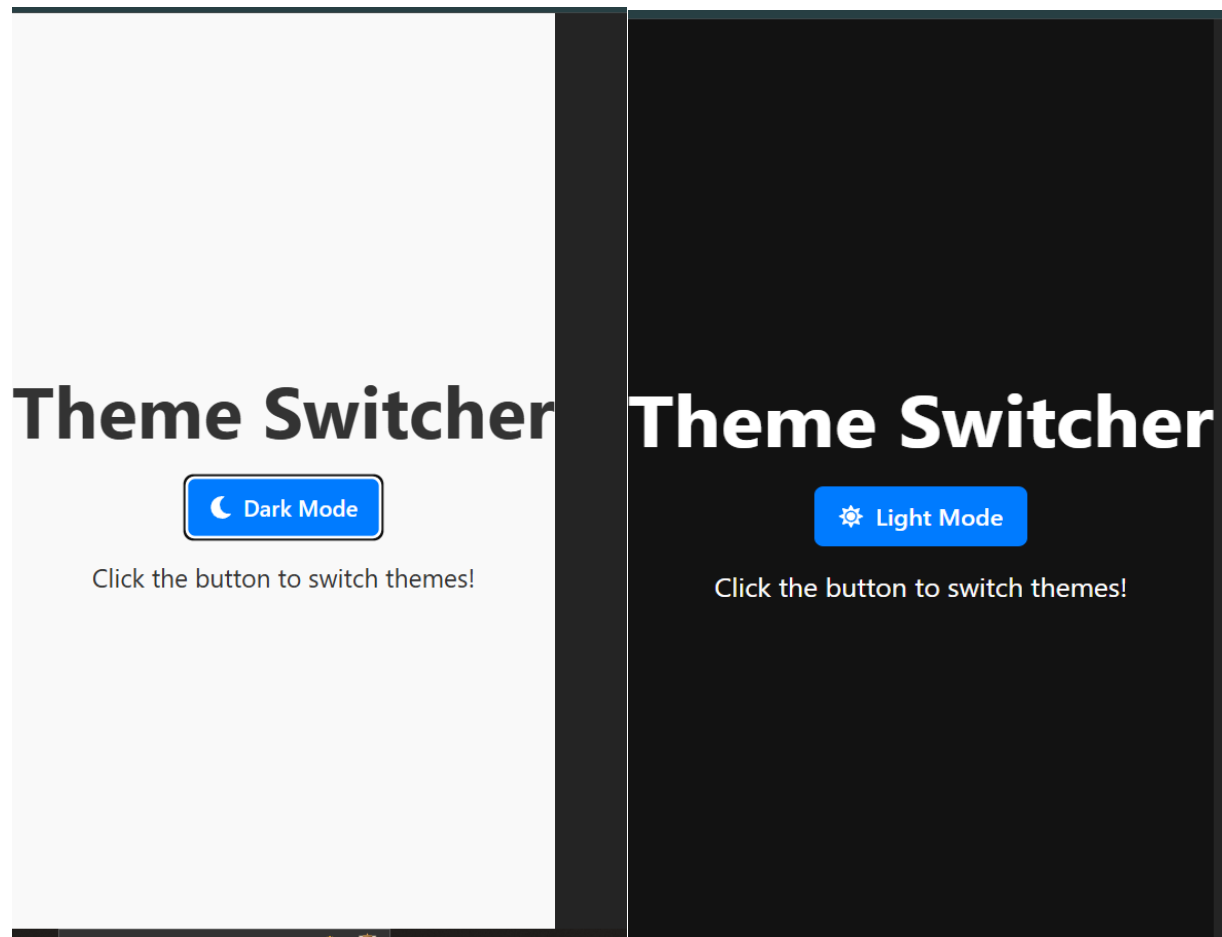
practical_18 > theme-switcher > src > contexts > ThemeContext.jsx > ThemeProvider > setTheme
1 import { createContext, useState, useContext } from "react";
2
3 // Create Theme Context
4 const ThemeContext = createContext();
5
6 // Custom Hook for using Theme Context
7 export const useTheme = () => useContext(ThemeContext);
8
9 // Theme Provider Component
10 export const ThemeProvider = ({ children }) => {
11   const [theme, setTheme] = useState("light");
12
13   const toggleTheme = () => {
14     setTheme((prevTheme) => (prevTheme === "light" ? "dark" : "light"));
15   };
16
17   return (
18     <ThemeContext.Provider value={{ theme, toggleTheme }}>
19       {children}
20     </ThemeContext.Provider>
21   );
22 };
23

```



The screenshot shows a code editor with four tabs: ThemeContext.jsx, App.jsx, main.jsx (active), and App.css. The main.jsx file contains the following code:

```
practical_18 > theme-switcher > src > main.jsx
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import App from "../App.jsx";
4  import { ThemeProvider } from "../contexts/ThemeContext";
5  import "../index.css";
6
7  ReactDOM.createRoot(document.getElementById("root")).render(
8    <React.StrictMode>
9      <ThemeProvider>
10        <App />
11      </ThemeProvider>
12    </React.StrictMode>
13  );
14
```

Screenshots of Output :

Conclusion: We created an app that can switch between light and dark themes. This teaches us how to share information across different parts of a React application and change how everything looks with one button.

Practical 19

Project: Weather App

Objective: Understand how to use the useEffect hook to fetch and display data from an external API.

Tasks:

- Fetch weather data for a user-input city using an API (e.g., Open Weather Map).
- Display the city name, temperature, and weather conditions.

Update the displayed data when the user searches for a new city.

Screenshots of Code:

```

App.tsx 9+ X
src > App.tsx > ...
 1  import React, { useState, useEffect } from "react";
 2  import "./styles.css";
 3
 4  function WeatherApp() {
 5    const [city, setCity] = useState("New York");
 6    const [weather, setWeather] = useState(null);
 7    const [error, setError] = useState(null);
 8
 9    useEffect(() => {
10      fetchWeather(city);
11    }, []);
12
13    const fetchWeather = async (cityName) => {
14      try {
15        const geoResponse = await fetch(
16          `https://geocoding-api.open-meteo.com/v1/search?name=${cityName}&count=1`
17        );
18        const geoData = await geoResponse.json();
19
20        if (!geoData.results || geoData.results.length === 0) {
21          throw new Error("City not found");
22        }
23

```

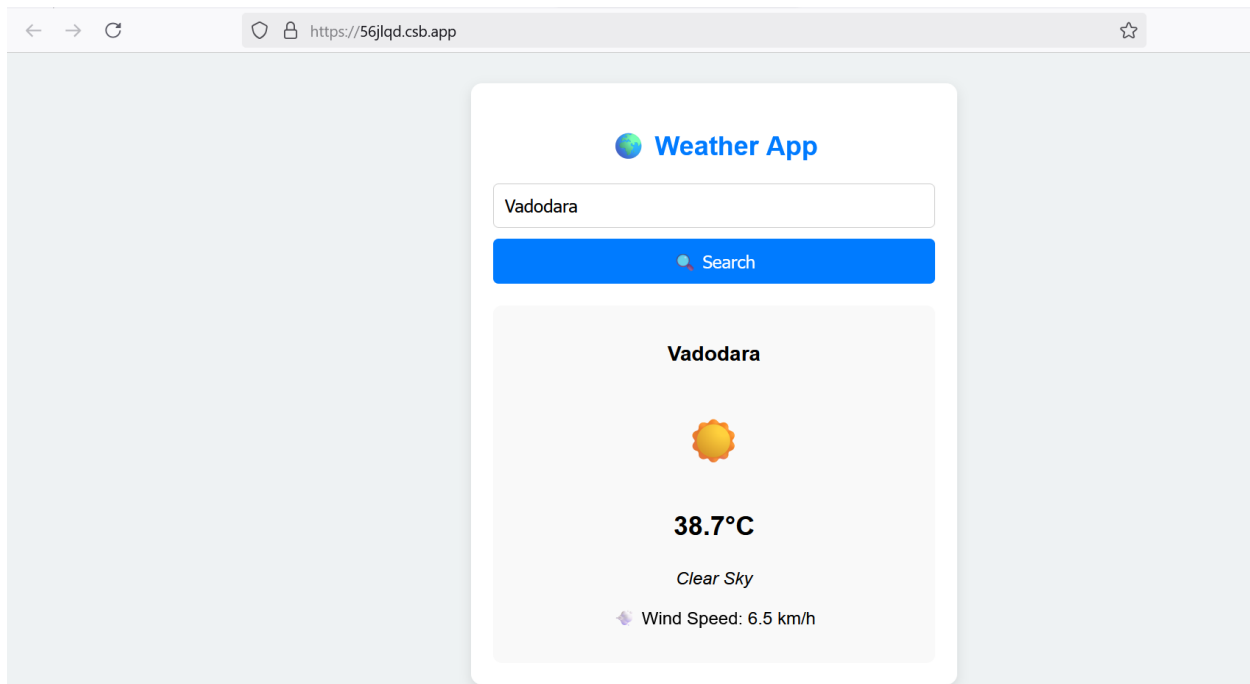
```

24    const { latitude, longitude, name } = geoData.results[0];
25
26    const weatherResponse = await fetch(
27      `https://api.open-meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}&current_weather=true`
28    );
29    const weatherData = await weatherResponse.json();
30
31    setWeather({
32      name,
33      temp: weatherData.current_weather.temperature,
34      windspeed: weatherData.current_weather.windspeed,
35      condition: getWeatherCondition(weatherData.current_weather.weathercode),
36      icon: getWeatherIcon(weatherData.current_weather.weathercode),
37    });
38    setError(null);
39  } catch (err) {
40    setError(err.message);
41    setWeather(null);
42  }
43  };

```

```
44
45   const getWeatherCondition = (code) => {
46     const conditions = {
47       0: "Clear Sky",
48       1: "Mainly Clear",
49       2: "Partly Cloudy",
50       3: "Overcast",
51       45: "Fog",
52       48: "Depositing Rime Fog",
53       51: "Light Drizzle",
54       53: "Moderate Drizzle",
55       55: "Dense Drizzle",
56       61: "Light Rain",
57       63: "Moderate Rain",
58       65: "Heavy Rain",
59       71: "Light Snow",
60       73: "Moderate Snow",
61       75: "Heavy Snow",
62       95: "Thunderstorms",
63     };
64     return conditions[code] || "Unknown Condition";
65   };
66
67   const getWeatherIcon = (code) => {
68     const icons = {
69       0: "☀️",
70       1: "☁️",
71       2: "☀️☁️",
72       3: "☁️",
73       45: "🌫️",
74       48: "❄️",
75       51: "🌧️",
76       53: "🌧️",
77       55: "🌧️",
78       61: "🌧️",
79       63: "🌧️",
80       65: "🌧️",
81       71: "🌨️",
82       73: "❄️",
83       75: "❄️",
84       95: "⚡️",
85     };
86     return icons[code] || "❓";
87   };
```

```
88
89   const handleSearch = () => {
90     if (city.trim() === "") return;
91     fetchWeather(city);
92   };
93
94   return (
95     <div className="container">
96       <h2>🌤 Weather App</h2>
97       <div className="input-group">
98         <input
99           type="text"
100           placeholder="Enter city"
101           value={city}
102           onChange={(e) => setCity(e.target.value)}
103         />
104         <button onClick={handleSearch}>🔍 Search</button>
105       </div>
106
107       {error && <p className="error">{error}</p>}
108
109       {weather && (
110         <div className="weather-card">
111           <h3>{weather.name}</h3>
112           <p className="weather-icon">{weather.icon}</p>
113           <p className="temperature">{weather.temp}°C</p>
114           <p className="condition">{weather.condition}</p>
115           <p>🌬 Wind Speed: {weather.windspeed} km/h</p>
116         </div>
117       )}
118     </div>
119   );
120 }
121
122 export default function App() {
123   return <WeatherApp />;
124 }
125
```

Screenshots of Output :

Conclusion: This practical showed us how to create an app that gets weather information from the internet and displays it to users. This helps understand how apps fetch and show real-time data.

Practical 20

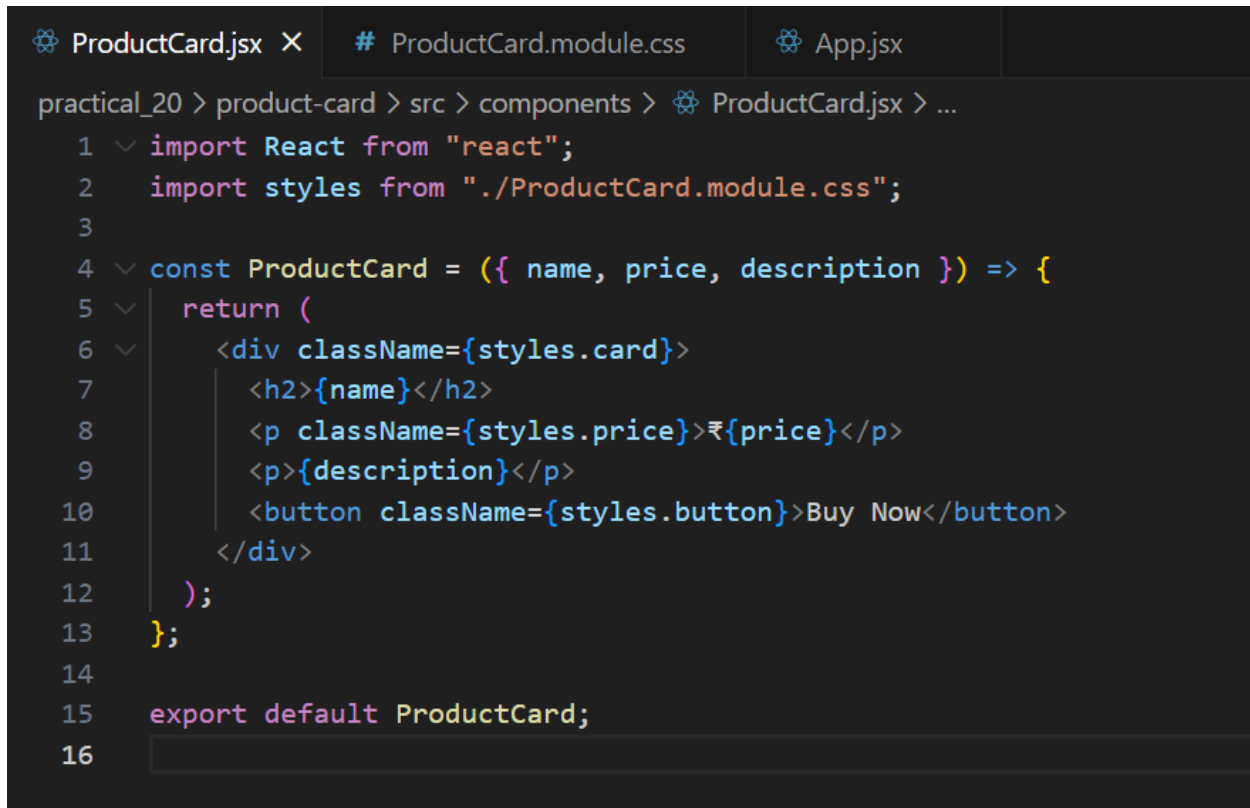
Project: Responsive Product Card

Objective: Learn how to style React components using modern CSS techniques.

Tasks:

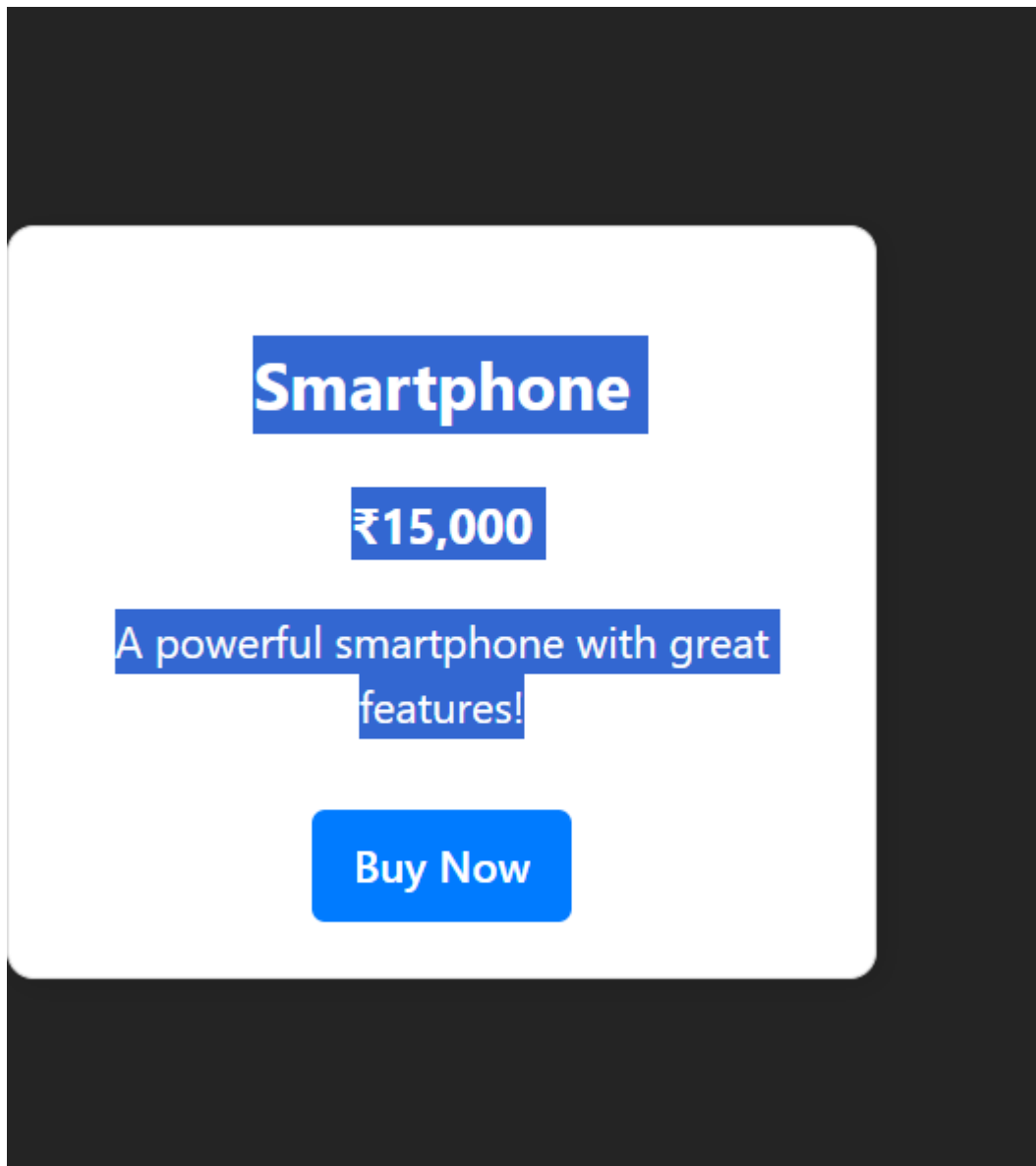
- Create a product card component with name, price, and description.
- Use CSS Modules to style the card.
- Add responsive styling to adjust layout for different screen sizes.

Screenshots of Code:



```
ProductCard.jsx × # ProductCard.module.css App.jsx
practical_20 > product-card > src > components > ProductCard.jsx > ...
1  import React from "react";
2  import styles from "../ProductCard.module.css";
3
4  const ProductCard = ({ name, price, description }) => {
5    return (
6      <div className={styles.card}>
7        <h2>{name}</h2>
8        <p className={styles.price}>₹{price}</p>
9        <p>{description}</p>
10       <button className={styles.button}>Buy Now</button>
11     </div>
12   );
13 };
14
15 export default ProductCard;
16
```

```
practical_20 > product-card > src > components > # ProductCard.module.css >
1  .card {
2      width: 280px;
3      padding: 20px;
4      border: 1px solid #ddd;
5      border-radius: 10px;
6      text-align: center;
7      background: white;
8      box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);
9      transition: transform 0.2s ease-in-out;
10 }
11
12 .card:hover {
13     transform: scale(1.05);
14 }
15
16 .price {
17     color: green;
18     font-weight: bold;
19     font-size: 18px;
20 }
21
22 .button {
23     margin-top: 10px;
24     padding: 10px 15px;
25     border: none;
26     background: #007bff;
27     color: white;
28     cursor: pointer;
29     border-radius: 5px;
30     font-size: 16px;
31 }
32
33 .button:hover {
34     background: #0056b3;
35 }
36
```

Screenshots of Output:

Conclusion: We built a product card that looks good on both big and small screens. This teaches us how to make websites that adapt to different device sizes.

Practical 21

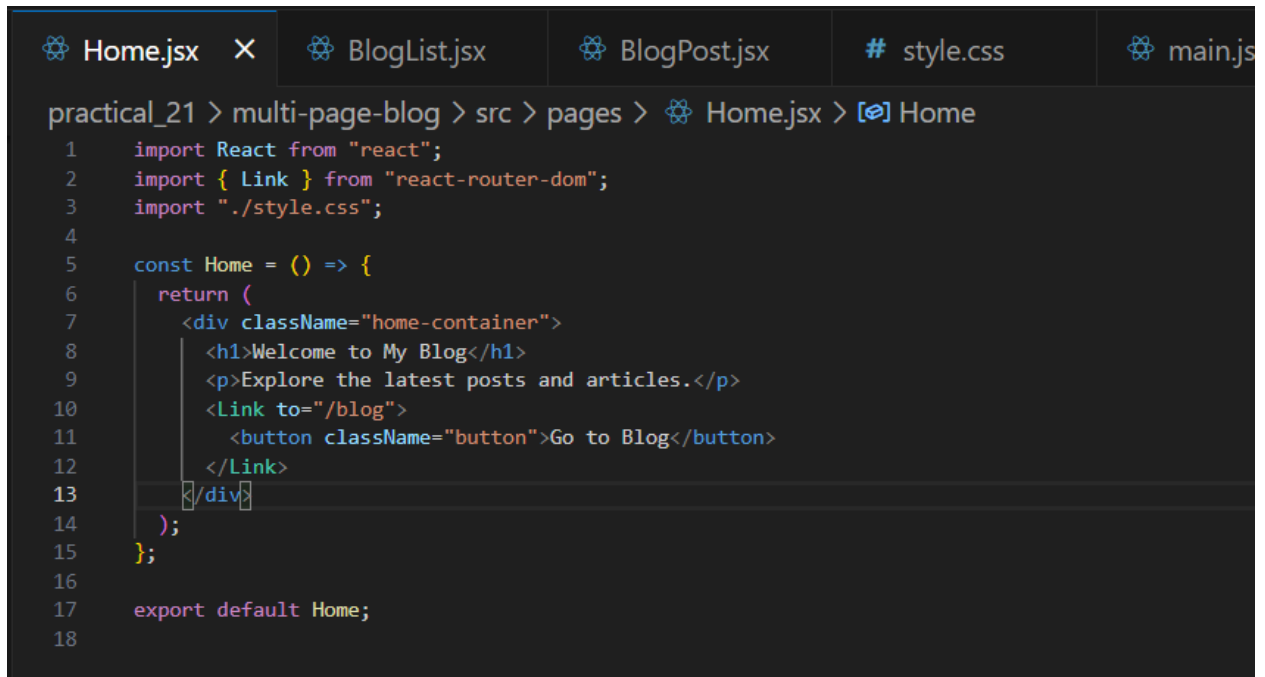
Project: Multi-Page Blog

Objective: Learn to build multi-page applications using React Router.

Tasks:

- Set up routes for a homepage, blog list, and individual blog post pages.
- Use React Router to navigate between pages.
- Display blog post details dynamically based on route parameters.

Screenshots of Code:



The screenshot shows a code editor with five tabs: Home.jsx, BlogList.jsx, BlogPost.jsx, style.css, and main.js. The Home.jsx tab is active, showing the following code:

```
practical_21 > multi-page-blog > src > pages > Home.jsx > Home
1  import React from "react";
2  import { Link } from "react-router-dom";
3  import "../style.css";
4
5  const Home = () => {
6    return (
7      <div className="home-container">
8        <h1>Welcome to My Blog</h1>
9        <p>Explore the latest posts and articles.</p>
10       <Link to="/blog">
11         <button className="button">Go to Blog</button>
12       </Link>
13     </div>
14   );
15 };
16
17 export default Home;
18
```

Home.jsx

BlogList.jsx X

BlogPost.jsx

style.css

main.jsx

practical_21 > multi-page-blog > src > pages > BlogList.jsx > blogs

```

1  import React from "react";
2  import { Link } from "react-router-dom";
3  import "../style.css";
4
5  const blogs = [
6    { id: 1, title: "Understanding React Hooks" },
7    { id: 2, title: "Introduction to React Router" },
8    { id: 3, title: "State Management in React" }
9  ];
10
11  const BlogList = () => {
12    return (
13      <div className="bloglist-container">
14        <h1>Blog Posts</h1>
15        <ul>
16          {blogs.map((blog) => (
17            <li key={blog.id}>
18              <Link to={`/blog/${blog.id}`} className="blog-link">
19                {blog.title}
20              </Link>
21            </li>
22          ))}
23        </ul>
24        <Link to="/">
25          <button className="button">Back to Home</button>
26        </Link>
27      </div>
28    );
29  };
30
31  export default BlogList;
32

```

Home.jsx

BlogList.jsx

BlogPost.jsx X

style.css

main.jsx

practical_21 > multi-page-blog > src > pages > BlogPost.jsx > ...

```

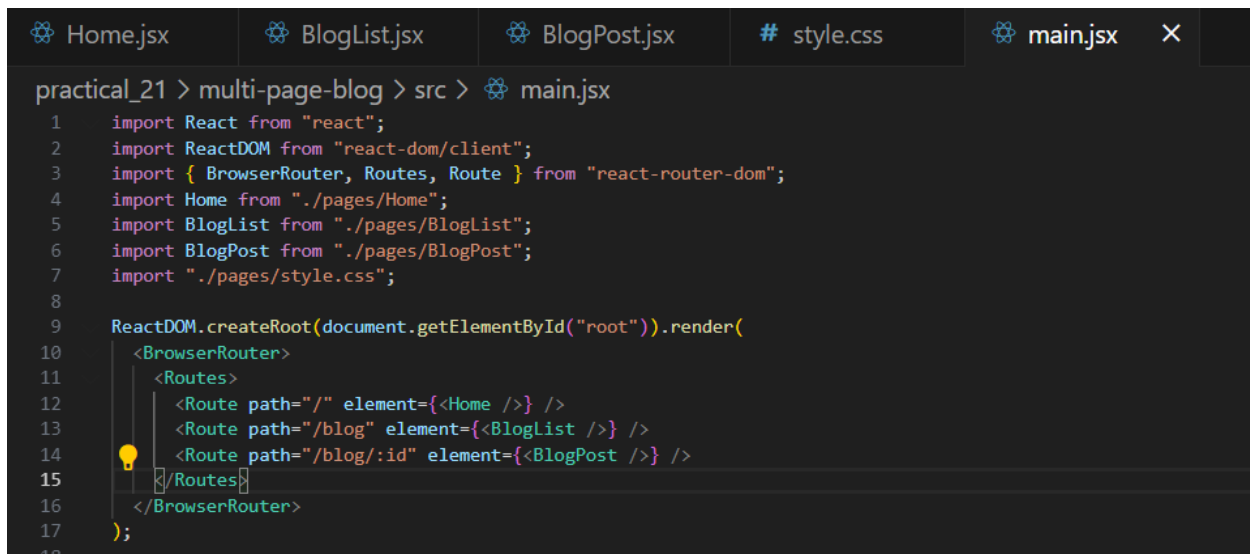
1  import React from "react";
2  import { useParams, Link } from "react-router-dom";
3  import "../style.css";
4
5  const blogData = {
6    1: { title: "Understanding React Hooks", content: "React Hooks allow functional components to use state and side effects." },
7    2: { title: "Introduction to React Router", content: "React Router helps in building single-page applications with multiple views." },
8    3: { title: "State Management in React", content: "Managing state in React can be done using useState, useReducer, or libraries like Redux." }
9  };
10
11  const BlogPost = () => {
12    const { id } = useParams();
13    const blog = blogData[id];
14
15    if (!blog) {
16      return <h2 className="not-found">Blog post not found!</h2>;
17    }
18
19    return (
20      <div className="blogpost-container">
21        <h1>{blog.title}</h1>
22        <p>{blog.content}</p>
23        <Link to="/blog">
24          <button className="button">Back to Blog List</button>
25        </Link>
26      </div>
27    );
28  };
29
30  export default BlogPost;
31

```



The image shows a code editor with four tabs at the top: Home.jsx, BlogList.jsx, BlogPost.jsx, and # style.css. The active tab is # style.css. The editor's breadcrumb path is 'practical_21 > multi-page-blog > src > pages > # style.css > ...'. The code is CSS for styling a multi-page blog. It includes general styling for the body, specific styling for the home page container, button styling with a hover effect, and styling for the blog list container and individual blog links. Line numbers 1 through 45 are visible on the left side of the code block.

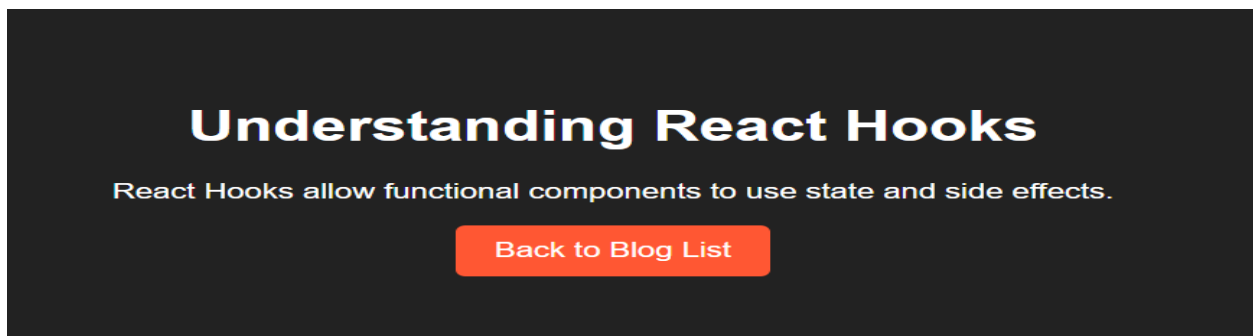
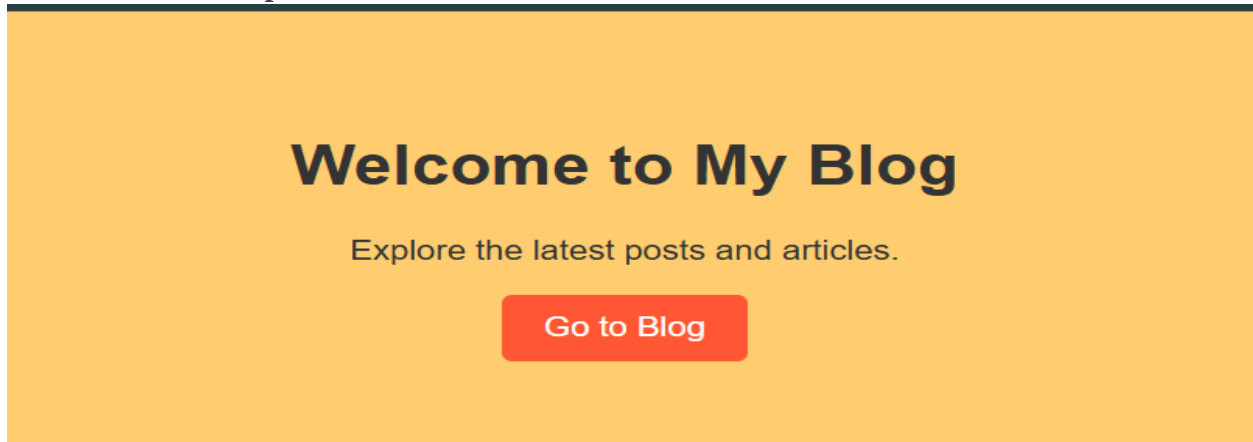
```
1  /* General Styling */
2  body {
3      font-family: "Arial", sans-serif;
4      margin: 0;
5      padding: 0;
6  }
7
8  /* Home Page */
9  .home-container {
10     background-color: #ffcc70;
11     color: #333;
12     text-align: center;
13     padding: 50px;
14 }
15
16 .button {
17     background-color: #ff5733;
18     color: white;
19     border: none;
20     padding: 10px 20px;
21     font-size: 16px;
22     cursor: pointer;
23     border-radius: 5px;
24     transition: background 0.3s;
25 }
26
27 .button:hover {
28     background-color: #d84315;
29 }
30
31 /* Blog List */
32 .bloglist-container {
33     background-color: #007bff;
34     color: white;
35     text-align: center;
36     padding: 50px;
37 }
38
39 .blog-link {
40     color: white;
41     text-decoration: none;
42     font-size: 18px;
43     display: block;
44     margin: 10px 0;
45 }
```



The screenshot shows a code editor with a dark theme. At the top, there are five tabs: 'Home.jsx', 'BlogList.jsx', 'BlogPost.jsx', '# style.css', and 'main.jsx'. The 'main.jsx' tab is active. The editor content shows the following code:

```
practical_21 > multi-page-blog > src > main.jsx
1  import React from "react";
2  import ReactDOM from "react-dom/client";
3  import { BrowserRouter, Routes, Route } from "react-router-dom";
4  import Home from "../pages/Home";
5  import BlogList from "../pages/BlogList";
6  import BlogPost from "../pages/BlogPost";
7  import "../pages/style.css";
8
9  ReactDOM.createRoot(document.getElementById("root")).render(
10    <BrowserRouter>
11      <Routes>
12        <Route path="/" element={<Home />} />
13        <Route path="/blog" element={<BlogList />} />
14        <Route path="/blog/:id" element={<BlogPost />} />
15      </Routes>
16    </BrowserRouter>
17  );
```

A yellow lightbulb icon is visible next to line 14, indicating a suggestion or warning. The cursor is positioned at the end of line 14.

Screenshots of Output:

Conclusion: This practical taught us how to create a blog with multiple pages that users can navigate between. This helps understand how to build websites with different sections and pages.

Practical 22

Project: Expense Tracker App

Objective: Learn to manage complex state logic with the useReducer hook.

Tasks:

- Set up a reducer function to manage a list of expenses.
- Add functionality to add and delete expense items.
- Display a summary of total expenses dynamically.

Screenshots of Code :-



```
1  import React, { useState } from "react";
2
3  const ExpenseForm = ({ dispatch }) => {
4    const [title, setTitle] = useState("");
5    const [amount, setAmount] = useState("");
6
7    const handleSubmit = (e) => {
8      e.preventDefault();
9      if (!title || !amount) return;
10
11      const newExpense = {
12        id: Date.now(),
13        title,
14        amount: parseFloat(amount),
15      };
16
17      dispatch({ type: "ADD_EXPENSE", payload: newExpense });
18
19      setTitle("");
20      setAmount("");
21    };
22
23    return (
24      <form className="expense-form" onSubmit={handleSubmit}>
25        <input
26          type="text"
27          placeholder="Expense Title"
28          value={title}
29          onChange={(e) => setTitle(e.target.value)}
30          required
31        />
32        <input
33          type="number"
34          placeholder="Amount (₹)"
35          value={amount}
36          onChange={(e) => setAmount(e.target.value)}
37          required
38        />
39        <button type="submit">Add Expense</button>
40      </form>
41    );
42  };
43
44  export default ExpenseForm;
45
```

```

App.jsx  ExpenseForm.jsx  ExpenseList.jsx  main.jsx  style.css
practical_22 > expense-tracker > src > components > ExpenseList.jsx > ...
1  import React from "react";
2
3  const ExpenseList = ({ expenses, dispatch }) => {
4    return (
5      <div className="expense-list">
6        <h3>Expenses</h3>
7        {expenses.length === 0 ? (
8          <p className="no-expense">No expenses added yet.</p>
9        ) : (
10          <ul>
11            {expenses.map((expense) => (
12              <li key={expense.id}>
13                <span>{expense.title}</span>
14                <span>{expense.amount}</span>
15                <button onClick={() => dispatch({ type: "DELETE_EXPENSE", payload: expense.id })}>
16                  ✕
17                </button>
18              </li>
19            )>
20          </ul>
21        )}
22      </div>
23    );
24  };
25
26  export default ExpenseList;
27

```

```

practical_22 > expense-tracker > src > components > #
1  /* General Styling */
2  * {
3    margin: 0;
4    padding: 0;
5    box-sizing: border-box;
6    font-family: "Arial", sans-serif;
7  }
8
9  body {
10   background: #f7f7f7;
11   display: flex;
12   justify-content: center;
13   align-items: center;
14   height: 100vh;
15 }
16
17 .app-container {
18   background: white;
19   padding: 30px;
20   width: 400px;
21   border-radius: 10px;
22   box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
23   text-align: center;
24 }
25
26 h1 {
27   color: #333;
28 }
29
30 h2 {
31   margin-top: 20px;
32   color: #007bff;
33 }
34
35 /* Expense Form */
36 .expense-form {
37   display: flex;
38   flex-direction: column;
39   gap: 10px;
40 }
41
42 .expense-form input {
43   padding: 10px;
44   border: 1px solid #ccc;
45   border-radius: 5px;
46
47   cursor: pointer;
48 }
49
50 .expense-form button:hover {
51   background: #0056b3;
52 }
53
54 /* Expense List */
55 .expense-list {
56   margin-top: 20px;
57 }
58
59 .expense-list ul {
60   list-style: none;
61   padding: 0;
62 }
63
64 .expense-list li {
65   display: flex;
66   justify-content: space-between;
67   background: #f8f9fa;
68   padding: 10px;
69   border-radius: 5px;
70   margin-bottom: 5px;
71   align-items: center;
72 }
73
74 .expense-list li span {
75   font-weight: bold;
76 }
77
78 .expense-list button {
79   background: red;
80   color: white;
81   border: none;
82   padding: 5px;
83   border-radius: 3px;
84   cursor: pointer;
85 }
86
87 .expense-list button:hover {
88   background: darkred;
89 }
90

```

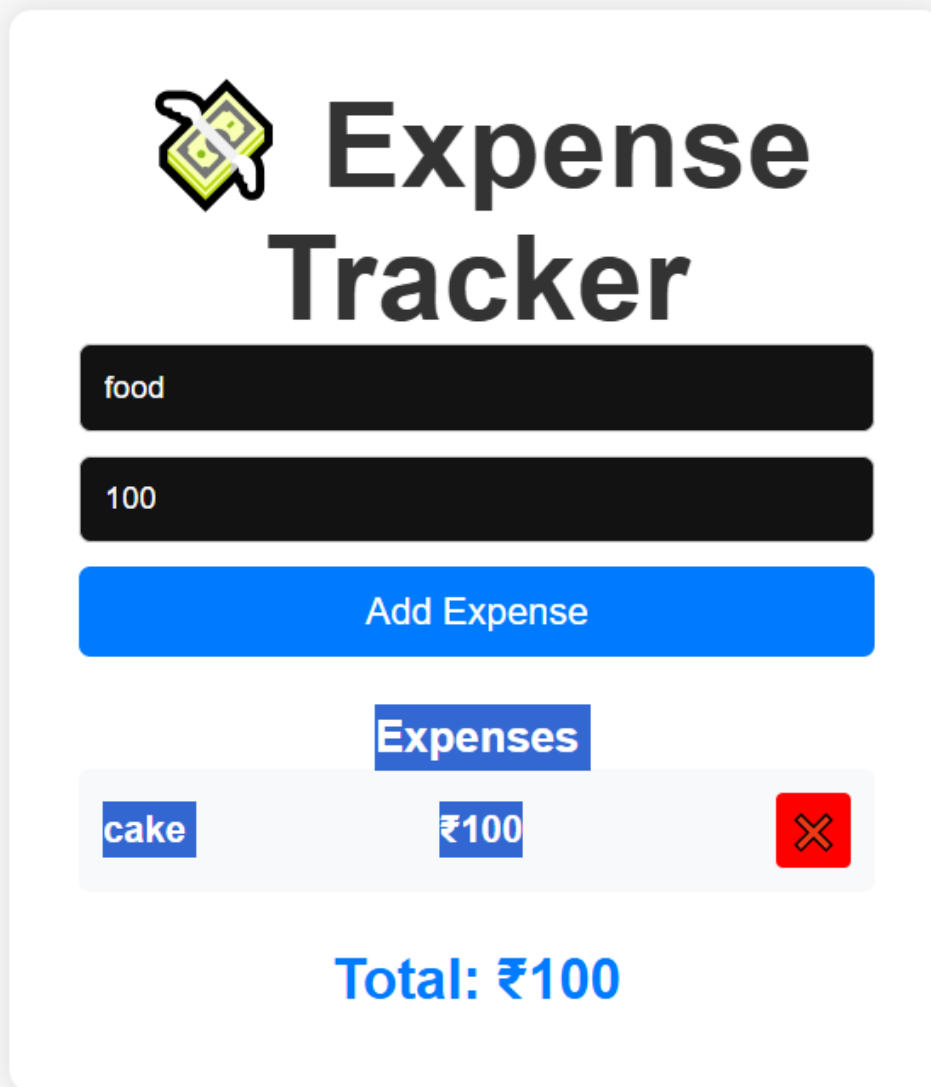
```

App.jsx  X  ExpenseForm.jsx  ExpenseList.jsx  main.jsx

practical_22 > expense-tracker > src > App.jsx > ...
1  import React, { useReducer } from "react";
2  import ExpenseForm from "../components/ExpenseForm";
3  import ExpenseList from "../components/ExpenseList";
4  import "../components/style.css";
5
6  // Initial State
7  const initialState = {
8    expenses: [],
9    total: 0,
10 };
11
12 // Reducer Function
13 const expenseReducer = (state, action) => {
14   switch (action.type) {
15     case "ADD_EXPENSE":
16       return {
17         expenses: [...state.expenses, action.payload],
18         total: state.total + action.payload.amount,
19       };
20     case "DELETE_EXPENSE":
21       const updatedExpenses = state.expenses.filter(
22         (expense) => expense.id !== action.payload
23       );
24       const updatedTotal = updatedExpenses.reduce((acc, item) => acc + item.amount, 0);
25       return { expenses: updatedExpenses, total: updatedTotal };
26     default:
27       return state;
28   }
29 };
30
31 const App = () => {
32   const [state, dispatch] = useReducer(expenseReducer, initialState);
33
34   return (
35     <div className="app-container">
36       <h1> Expense Tracker</h1>
37       <ExpenseForm dispatch={dispatch} />
38       <ExpenseList expenses={state.expenses} dispatch={dispatch} />
39       <h2>Total: ₹{state.total}</h2>
40     </div>
41   );
42 };
43
44 export default App;
45

```

Screenshots of Output:



The screenshot displays the 'Expense Tracker' application interface. At the top, there is a logo consisting of a yellow square with a black 'X' and a green square with a white 'X'. Below the logo, the title 'Expense Tracker' is written in a large, bold, black font. Underneath the title, there are two input fields: the first contains the text 'food' and the second contains the number '100'. Below these fields is a blue button with the text 'Add Expense'. Below the button is a section titled 'Expenses' in a blue box. Under this title, there is a list of expenses. The first item is 'cake' in a blue box, followed by '₹100' in a blue box, and a red square button with a white 'X' icon. Below the list of expenses, the total amount is displayed as 'Total: ₹100' in a large, bold, blue font.

Conclusion: We created an app that helps track expenses, allowing users to add and delete expense items and see the total amount spent. This teaches us how to manage more complex information in React applications.

Practical 23

Project: Testing a Task Manager App

Objective: Test the key features of a "Task Manager" app for managing tasks.

Backend Testing:

- Setup: Use Jest and Super test to test the Node.js backend API.
- Endpoints to Test:

1. POST /tasks: Test if a task is created with valid data.
2. GET /tasks: Test if all tasks are returned.
3. PUT /tasks/:id: Test if a task is updated correctly.
4. DELETE /tasks/:id: Test if a task is deleted by ID.

Screenshots of Code:

```
test > JS server.js > ...
1  require("dotenv").config();
2  const express = require("express");
3  const mongoose = require("mongoose");
4  const taskRoutes = require("./routes/taskRoutes");
5
6  const app = express();
7  app.use(express.json());
8  app.use("/tasks", taskRoutes);
9
10 mongoose
11   .connect(process.env.MONGO_URI, { useNewUrlParser: true, useUnifiedTopology: true })
12   .then(() => console.log("MongoDB Connected"))
13   .catch((err) => console.log(err));
14
15 const PORT = process.env.PORT || 5000;
16 app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
17
18 module.exports = app; // Exporting for testing
19
```

```
test > .env
1  MONGO_URI=mongodb://localhost:27017/taskmanager2
2  PORT=5000
```

```
test > models > JS Task.js > ...
1  const mongoose = require("mongoose");
2
3  const TaskSchema = new mongoose.Schema({
4    title: { type: String, required: true },
5    completed: { type: Boolean, default: false }
6  });
7
8  module.exports = mongoose.model("Task", TaskSchema);
9
```

```
test > routes > JS taskRoutes.js > ...
1  const express = require("express");
2  const Task = require("../models/Task");
3  const router = express.Router();
4  router.post("/", async (req, res) => {
5    try {
6      const task = new Task(req.body);
7      await task.save();
8      res.status(201).json(task);
9    } catch (error) {
10     res.status(400).json({ error: error.message });
11   }
12 });
13 router.get("/", async (req, res) => {
14   const tasks = await Task.find();
15   res.json(tasks);
16 });
17 router.put("/:id", async (req, res) => {
18   try {
19     const task = await Task.findByIdAndUpdate(req.params.id, req.body, { new: true });
20     res.json(task);
21   } catch (error) {
22     res.status(400).json({ error: error.message });
23   }
24 });
25 router.delete("/:id", async (req, res) => {
26   try {
27     await Task.findByIdAndDelete(req.params.id);
28     res.status(200).json({ message: "Task deleted" });
29   } catch (error) {
30     res.status(400).json({ error: error.message });
31   }
32 });
33 module.exports = router;
34
```

```
test > tests > task.test.js > ...
1  const request = require("supertest");
2  const app = require("../server");
3  const mongoose = require("mongoose");
4  const Task = require("../models/Task");
5
6  beforeAll(async () => {
7    await Task.deleteMany(); // Clear DB before running tests
8  });
9
10 afterAll(async () => {
11   await mongoose.connection.close(); // Close DB connection
12 });
13
14 // Test POST /tasks
15 test("Should create a new task", async () => {
16   const response = await request(app).post("/tasks").send({ title: "Test Task" });
17   expect(response.statusCode).toBe(201);
18   expect(response.body.title).toBe("Test Task");
19 });
20
21 // Test GET /tasks
22 test("Should fetch all tasks", async () => {
23   const response = await request(app).get("/tasks");
24   expect(response.statusCode).toBe(200);
25   expect(response.body.length).toBeGreaterThan(0);
26 });
27
28 // Test PUT /tasks/:id
29 test("Should update a task", async () => {
30   const task = await Task.create({ title: "Old Task" });
31   const response = await request(app).put(`/tasks/${task._id}`).send({ title: "Updated Task" });
32   expect(response.statusCode).toBe(200);
33   expect(response.body.title).toBe("Updated Task");
34 });
```

```
// Test DELETE /tasks/:id
test("Should delete a task", async () => {
  const task = await Task.create({ title: "To be deleted" });
  const response = await request(app).delete(`/tasks/${task._id}`);
  expect(response.statusCode).toBe(200);
});
```

Screenshots of Output:

```
PS D:\fswd\test> node server.js
(node:3836) [MONGODB DRIVER] Warning: useNewUrlParser
be removed in the next major version
(Use `node --trace-warnings ...` to show where the warn
(node:3836) [MONGODB DRIVER] Warning: useUnifiedTopolo
will be removed in the next major version
Server running on port 5000
MongoDB Connected
PS D:\fswd\test> npm test
>>

> test@1.0.0 test
> jest
```

```
console.log
  Server running on port 5000

    at Server.log (server.js:16:32)

console.log
  MongoDB Connected

    at log (server.js:12:23)
```

```
PASS tests/task.test.js
  ✓ Should create a new task (42 ms)
  ✓ Should fetch all tasks (11 ms)
PASS tests/task.test.js
  ✓ Should create a new task (42 ms)
PASS tests/task.test.js
PASS tests/task.test.js
  ✓ Should create a new task (42 ms)
  ✓ Should fetch all tasks (11 ms)
  ✓ Should update a task (17 ms)
  ✓ Should delete a task (10 ms)
```

```
Test Suites: 1 passed, 1 total
Tests:       4 passed, 4 total
Snapshots:   0 total
Time:        2.86 s, estimated 3 s
Ran all test suites.
```

```
Jest did not exit one second after the test run has completed.
```

Conclusion: This practical showed us how to test a task manager app to make sure it works correctly. We learned how to check if the app can create, show, update, and delete tasks properly. This helps understand how to make sure software works as expected.

Github Link:

<https://github.com/Bobbymkr/Kunj/tree/FSWD-Practicals>